

<PROJECT PDF DUMP>
Generated at: 2026-04-24T16:16:12.036556 UTC
Root: C:\Users\morty\video-exchange-bot
Files included: 16

```
=====
FILE: .env
=====
```

```
BOT_TOKEN=8747618457:AAHz0LNqPr4wnSW0dDhT0xLwyREc-JKHINI
ADMINS=5272076117
WEBHOOK_BASE=https://placeholder.onrender.com
WEBHOOK_PATH=/webhook
DATABASE_URL=sqlite+aiosqlite:///bot.db
PORT=10000
```

```
=====
FILE: .env.example
=====
```

```
BOT_TOKEN=123456:ABC-DEF
ADMINS=123456789
WEBHOOK_BASE=https://your-app.onrender.com
WEBHOOK_PATH=/webhook
DATABASE_URL=postgresql+asyncpg://user:password@host:5432/dbname
```

```
=====
FILE: app\__init__.py
=====
```

```
=====
FILE: app\admin_handlers.py
=====
```

```
from datetime import datetime, timedelta
from decimal import Decimal
from io import BytesIO

from aiogram import Router, F
from aiogram.filters import Command
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.context import FSMContext
from aiogram.types import (
    Message, CallbackQuery,
    InlineKeyboardMarkup, InlineKeyboardButton
)
from sqlalchemy import select, func, desc

from app.config import ADMINS, OFFER_DEFAULT_RENT_COST_PER_DAY
from app.db import async_session
from app.models import (
    User, Video, Offer, BalanceLog, GameHistory,
    UserActionLog, OfferRental, OfferParticipation
)
from app.services import (
    get_user, get_user_by_id, get_user_by_username,
    get_user_dossier, update_user_balance, set_user_ban_status,
    count_pending_videos, count_approved_videos, count_rejected_videos,
    get_next_pending_video, approve_video, reject_video,
    get_admin_extended_stats, to_decimal, get_display_name,
    get_user_by_display_name, admin_create_offer,
    count_active_rentals, expire_old_rentals, log_user_action,
)
from app.keyboards import (
    admin_main_keyboard, moderation_keyboard,
    rejection_reason_keyboard, admin_after_action_keyboard,
    admin_offers_keyboard,
)

router = Router()

# =====
# STATES
# =====
class AdminUserState(StatesGroup):
    waiting_user_id = State()
    waiting_coins_amount = State()
    waiting_message_text = State()
    waiting_dossier_id = State()

class AdminManageState(StatesGroup):
    waiting_new_admin = State()
    waiting_remove_admin = State()

class AdminBroadcastState(StatesGroup):
    waiting_text = State()
```

```

class AdminNicknameState(StatesGroup):
    waiting_user_id = State()
    waiting_new_nick = State()

class AdminOfferCreateState(StatesGroup):
    waiting_title = State()
    waiting_description = State()
    waiting_url = State()
    waiting_reward_preview = State()
    waiting_reward_final = State()
    waiting_rentable = State()
    waiting_rent_cost = State()
    waiting_max_rentals = State()

# =====
# HELPERS
# =====
def is_super_admin(tid: int) -> bool:
    return tid in ADMINS

async def check_admin(tid: int) -> bool:
    if tid in ADMINS:
        return True
    async with async_session() as session:
        user = await get_user(session, tid)
        if user and user.is_admin:
            return True
    return False

async def _safe_edit(callback: CallbackQuery, text: str, **kwargs):
    try:
        await callback.message.edit_text(text, **kwargs)
    except Exception:
        await callback.message.answer(text, **kwargs)

# =====
# ADMIN PANEL
# =====
@router.message(Command("admin"))
async def cmd_admin(message: Message):
    if not await check_admin(message.from_user.id):
        return
    sa = is_super_admin(message.from_user.id)
    await message.answer(
        "■■■ <b>■■■■■-■■■■■</b>",
        parse_mode="HTML",
        reply_markup=admin_main_keyboard(is_super=sa)
    )

@router.callback_query(F.data == "admin_center")
async def admin_center(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    sa = is_super_admin(callback.from_user.id)
    await _safe_edit(
        callback,
        "■■■ <b>■■■■■-■■■■■</b>",
        parse_mode="HTML",
        reply_markup=admin_main_keyboard(is_super=sa)
    )
    await callback.answer()

@router.callback_query(F.data == "admin_back")
async def admin_back(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    sa = is_super_admin(callback.from_user.id)
    await _safe_edit(
        callback,
        "■■■ <b>■■■■■-■■■■■</b>",
        parse_mode="HTML",
        reply_markup=admin_main_keyboard(is_super=sa)
    )
    await callback.answer()

# =====
# QUEUE INFO

```

```
# =====
@router.callback_query(F.data == "admin_queue_info")
async def cb_queue(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        p = await count_pending_videos(session)
        a = await count_approved_videos(session)
        r = await count_rejected_videos(session)

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ██████████", callback_data="admin_get_pending")],
        [InlineKeyboardButton(text="■ ██████████ ████", callback_data="admin_approve_all")],
        [InlineKeyboardButton(text="■ ██████", callback_data="admin_center")]
    ])
    await _safe_edit(
        callback,
        f"■ <b>██████████</b>\n\n"
        f"■ ███ ██████████: <b>{p}</b>\n"
        f"■ ██████████: <b>{a}</b>\n"
        f"■ ██████████: <b>{r}</b>",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()
```

```
# =====
# EXTENDED STATS
# =====
@router.callback_query(F.data == "admin_extended_stats")
async def admin_extended_stats(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        stats = await get_admin_extended_stats(session)

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ██████", callback_data="admin_center")]
    ])
    text = (
        f"■ <b>████████████████████ ████████████████████</b>\n\n"
        f"■ ████████████████████: <b>{stats['users']}</b>\n"
        f"■ VIP: <b>{stats['vip']}</b>\n"
        f"■ █ ████████: <b>{stats['with_nickname']}</b>\n"
        f"■ ████████████████████: <b>{stats['comments']}</b>\n"
        f"■ ♥ ██████████: <b>{stats['reactions']}</b>\n"
        f"■ ████: <b>{stats['games']}</b>\n"
        f"■ ██████████: <b>{stats['offers']}</b>\n"
        f"■ ████████████████████: <b>{stats.get('active_rentals', 0)}</b>\n\n"
        f"■ ██████████ ██████████: <b>{stats['total_balance_in_system']:.2f}</b>\n"
        f"■ ██████████ ██████████: <b>{stats['total_admin_given']:.2f}</b>\n"
        f"■ ██████████ ██████████: <b>{stats['total_game_profit']:.2f}</b>\n"
        f"■ ██████████ ██████████: <b>{abs(float(stats.get('total_rent_income', 0))):.2f}</b>"
    )
    await _safe_edit(callback, text, parse_mode="HTML", reply_markup=kb)
    await callback.answer()
```

```
# =====
# INVESTIGATION
# =====
@router.callback_query(F.data == "admin_investigation")
async def admin_investigation(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text="■ ████████████████████ ██████",
            callback_data="inv_suspicious_games"
        )],
        [InlineKeyboardButton(
            text="■ ███ ██████████ (██████████)",
            callback_data="inv_rich_detail"
        )],
        [InlineKeyboardButton(
            text="■ ████████████████████ ██████████",
            callback_data="inv_balance_log"
        )],
        [InlineKeyboardButton(
            text="■ ██████████ █████ ID/██████",
            callback_data="admin_user_dossier"
        )],
        [InlineKeyboardButton(
```



```

        f"■ <code>{u.telegram_id}</code>\n\n"
    )

kb = InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_investigation")]
])
if len(text) > 4000:
    text = text[:4000] + "\n..."
await callback.message.answer(text, parse_mode="HTML", reply_markup=kb)
await callback.answer()

@router.callback_query(F.data == "inv_balance_log")
async def inv_balance_log(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        rows = (await session.execute(
            select(BalanceLog, User)
            .join(User, User.id == BalanceLog.user_id)
            .order_by(desc(BalanceLog.created_at))
            .limit(30)
        )).all()

    if not rows:
        await callback.message.answer("■■■ ■■■■.")
        await callback.answer()
        return

    text = "■ <b>■■■■■■■■■■ 30 ■■■■■■■■ ■■■■■■■■</b>\n\n"
    for log, u in rows:
        sign = "+" if log.amount >= 0 else "-"
        text += (
            f"{log.created_at.strftime('%d.%m %H:%M')} | "
            f"<b>{get_display_name(u)}</b> | "
            f"{sign}{log.amount:.2f} [{log.source}]\n"
            f"    {log.balance_before:.2f} → {log.balance_after:.2f}"
        )
        if log.details:
            text += f"\n    ■ {log.details[:60]}"
        text += "\n\n"

    if len(text) > 4000:
        text = text[:4000] + "\n..."

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_investigation")]
    ])
    await callback.message.answer(text, parse_mode="HTML", reply_markup=kb)
    await callback.answer()

@router.callback_query(F.data == "inv_export_ai")
async def inv_export_ai(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        rich_users = (await session.execute(
            select(User).order_by(desc(User.balance)).limit(10)
        )).scalars().all()

        sus_rows = (await session.execute(
            select(
                User,
                func.sum(GameHistory.result).label("profit"),
                func.count(GameHistory.id).label("games")
            )
            .join(GameHistory, GameHistory.user_id == User.id)
            .group_by(User.id)
            .having(func.sum(GameHistory.result) > 30)
            .order_by(desc("profit"))
            .limit(10)
        )).all()

        admin_rows = (await session.execute(
            select(BalanceLog, User)
            .join(User, User.id == BalanceLog.user_id)
            .where(BalanceLog.source == "admin_balance")
            .order_by(desc(BalanceLog.created_at))
            .limit(20)
        )).all()

    try:
        rental_rows = (await session.execute(
            select(OfferRental, User)
            .join(User, User.id == OfferRental.renter_user_id)

```



```

caption = (
    f"■ #{video.id} | {video.content_type}\n"
    f"■ {name} (tg: {tg_id})\n"
    f"■ {video.created_at.strftime('%d.%m.%Y %H:%M')}\n"
    f"■ {video.duration_seconds or '?'} ■■ | "
    f"■ {round(video.file_size / 1024 / 1024, 2) if video.file_size else '?'} ■■"
)

try:
    if video.content_type == "photo":
        await callback.message.answer_photo(
            video.telegram_file_id,
            caption=caption,
            reply_markup=moderation_keyboard(video.id)
        )
    else:
        await callback.message.answer_video(
            video.telegram_file_id,
            caption=caption,
            reply_markup=moderation_keyboard(video.id)
        )
except Exception as e:
    await callback.message.answer(
        f"■■ ■■ ■■■■■■■■ ■■■■■■■■ ■■■■■: {e}\n{caption}",
        reply_markup=moderation_keyboard(video.id)
    )
await callback.answer()

@router.callback_query(F.data.startswith("mod_approve:"))
async def mod_approve(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    video_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        video = await approve_video(session, video_id)
    if not video:
        await callback.answer("■■■ ■■■■■■■■.", show_alert=True)
        return
    uploader = await get_user_by_id(session, video.uploader_user_id)
    if uploader:
        try:
            await callback.bot.send_message(
                uploader.telegram_id,
                f"■ ■■■■ ■■■■ #{video_id} ■■■■■■■■! ■■■■■ ■■■■■■■■."
            )
        except Exception:
            pass

# ■■■■■■■■■■ ■■■■■■■■ ■ ■■■■■, ■■■■■■■■ ■■■■■■■■
try:
    await callback.message.edit_caption(
        caption=f"■ #{video_id} - ■■■■■■■■",
        reply_markup=admin_after_action_keyboard()
    )
except Exception:
    try:
        await callback.message.edit_reply_markup(
            reply_markup=admin_after_action_keyboard()
        )
    except Exception:
        pass
await callback.answer(f"■ #{video_id} ■■■■■■■■!", show_alert=False)

@router.callback_query(F.data.startswith("mod_reject:"))
async def mod_reject(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    video_id = int(callback.data.split(":")[1])
    # ■■■■■■■■■■ ■■■■■■■■ ■■■■■ ■ inline-■■■■■■■■■■ ■■■ ■■■■■
    try:
        await callback.message.edit_reply_markup(
            reply_markup=rejection_reason_keyboard(video_id)
        )
    except Exception:
        await callback.message.answer(
            f"■■■■■■■■ ■■■■■■■■■■ #{video_id}:",
            reply_markup=rejection_reason_keyboard(video_id)
        )
    await callback.answer()

@router.callback_query(F.data.startswith("reject_reason:"))
async def reject_reason(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):

```



```

        result_text,
        parse_mode="HTML",
        reply_markup=kb
    )

# =====
# USER MANAGEMENT
# =====
@router.callback_query(F.data == "admin_manage_users")
async def admin_manage_users(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_user_dossier")],
        [InlineKeyboardButton(text="■ ■■■■■ ■■■■■", callback_data="admin_give_coins")],
        [InlineKeyboardButton(text="■ ■■■■■■■■■■■", callback_data="admin_ban_user")],
        [InlineKeyboardButton(text="■ ■■■■■■■■■■■", callback_data="admin_unban_user")],
        [InlineKeyboardButton(text="✉ ■■■■■■■■■", callback_data="admin_message_user")],
        [InlineKeyboardButton(text="■ ■■■■■■■■■", callback_data="admin_change_nickname")],
        [InlineKeyboardButton(text="■ ■■■■■■■", callback_data="admin_broadcast")],
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_center")],
    ])
    await _safe_edit(
        callback,
        "■ <b>■■■■■■■■■■ ■■■■■■■■■■■</b>",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()

# --- DOSSIER ---
@router.callback_query(F.data == "admin_user_dossier")
async def admin_user_dossier_start(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    await state.set_state(AdminUserState.waiting_dossier_id)
    await callback.message.answer(
        "■■■■■■ Telegram ID, @username ■■■ ■■■ ■■■■■■■■■■■:"
    )
    await callback.answer()

@router.message(AdminUserState.waiting_dossier_id)
async def process_dossier(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    query = message.text.strip()
    async with async_session() as session:
        user = None
        if query.startswith("@"):
            user = await get_user_by_username(session, query)
        elif query.isdigit():
            user = await get_user(session, int(query))
        else:
            user = await get_user_by_display_name(session, query)
            if not user:
                user = await get_user_by_username(session, query)

    if not user:
        await message.answer("■ ■■■■■■■■■■■ ■■ ■■■■■.")
        await state.clear()
        return

    dossier = await get_user_dossier(session, user.id)
    if not dossier:
        await message.answer("■ ■■ ■■■■■■■ ■■■■■■■■■■■.")
        await state.clear()
        return

    u = dossier["user"]
    suspicion = []
    if dossier["game_profit"] > 100:
        suspicion.append(f"■■■■■■■■■■■■■■■■■■■■: {dossier['game_profit']:.2f}")
    if dossier["admin_given"] > 200:
        suspicion.append(f"■■■■■■ ■■ ■■■■■■■■■■■■■■■■■■■: {dossier['admin_given']:.2f}")
    if dossier["suspicious_games"]:
        suspicion.append(f"■■■■■■■■■■. ■■■■■■■ ■■■■: {len(dossier['suspicious_games'])}")

    logs_text = ""
    for log in dossier["logs"][:5]:
        logs_text += f" • {log.action} ({log.created_at.strftime('%d.%m %H:%M')})\n"
        if log.details:
            logs_text += f" {str(log.details)[:50]}\n"

```

```

balance_logs_text = ""
for bl in dossier["balance_logs"][:5]:
    sign = "+" if bl.amount >= 0 else "-"
    balance_logs_text += (
        f" {bl.created_at.strftime('%d.%m %H:%M')} "
        f"{sign}{bl.amount:.2f} [{bl.source}] "
        f"{bl.balance_before:.2f}→{bl.balance_after:.2f}\n"
    )

text = (
    f"■ <b>■■■■■: {get_display_name(u)}</b>\n\n"
    f"■ TG: <code>{u.telegram_id}</code>\n"
    f"■ ■■■: {u.display_name or '■■ ■■■■'}\n"
    f"■ ■■■: {u.first_name or '???'}\n"
    f"■ @{u.username or '???'}\n"
    f"■ ■■■■■: <b>{u.balance:.2f}</b>\n"
    f"■ ■■. {u.level} | XP: {u.xp}\n"
    f"■ ■■■■■: {u.status}\n"
    f"■ VIP: {'■■' if u.vip_until and u.vip_until > datetime.utcnow() else '■■■'}\n"
    f"■ ■■■: {u.created_at.strftime('%d.%m.%Y')}\n\n"
    f"■ ■■■■■■■: {dossier['videos_uploaded']}\n"
    f"■ ■■■■■■■: {dossier['videos_watched']}\n"
    f"■ ■■■: {dossier['games_count']} | "
    f"■■■■■■: {dossier['game_profit']:.2f}\n"
    f"■ ■■■■ ■■■■■■■: {dossier['total_earned']:.2f}\n"
    f"■ ■■■■ ■■■■■■■: {abs(float(dossier['total_spent'])):.2f}\n"
    f"■ ■■ ■■■■■■■: {dossier['admin_given']:.2f}\n\n"
    f"■ <b>■■■■■:</b>\n"
    f"{''; '.join(suspicion) if suspicion else '■■■■■■■■■■ ■■■'}\n\n"
    f"■ <b>■■■■■■■■■■:</b>\n{logs_text or '■■■■'}\n"
    f"■ <b>■■■ ■■■■■■■:</b>\n{balance_logs_text or '■■■■'}"
)

kb = InlineKeyboardMarkup(inline_keyboard=[
    [
        InlineKeyboardButton(
            text="■ ■■■■■■■",
            callback_data=f"give_coins_to:{u.id}"
        ),
        InlineKeyboardButton(
            text="■ ■■■",
            callback_data=f"ban_user:{u.id}"
        ),
    ],
    [
        InlineKeyboardButton(
            text="■ ■■■■■■■",
            callback_data=f"unban_user:{u.id}"
        ),
        InlineKeyboardButton(
            text="=■ ■■■",
            callback_data=f"admin_set_nick:{u.id}"
        ),
    ],
    [InlineKeyboardButton(
        text="■ ■■■■■■■ ■■■ ■■■■■■■",
        callback_data=f"full_balance_log:{u.id}"
    )],
])

if len(text) > 4000:
    text = text[:4000] + "\n..."
await message.answer(text, parse_mode="HTML", reply_markup=kb)
await state.clear()

```

```

@router.callback_query(F.data.startswith("full_balance_log:"))
async def full_balance_log(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    user_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        u = await get_user_by_id(session, user_id)
        if not u:
            await callback.answer("■■ ■■■■■■■.", show_alert=True)
            return
        logs = (await session.execute(
            select(BalanceLog)
            .where(BalanceLog.user_id == user_id)
            .order_by(desc(BalanceLog.created_at))
            .limit(100)
        )).scalars().all()

    report = (
        f"=== ■■■ ■■■■■■■: {get_display_name(u)} "
        f"(tg={u.telegram_id}) ===\n"
        f"■■■■■■■■ ■■■■■■■: {u.balance}\n\n"
    )

```

```

    )
    for log in logs:
        sign = "+" if log.amount >= 0 else "-"
        report += (
            f"{log.created_at.strftime('%Y-%m-%d %H:%M:%S')} | "
            f"{sign}{log.amount} | {log.source} | "
            f"{log.balance_before}→{log.balance_after}"
        )
        if log.admin_id:
            report += f" | admin={log.admin_id}"
        if log.details:
            report += f" | {log.details}"
        report += "\n"

    buf = BytesIO(report.encode("utf-8"))
    buf.name = f"balance_{u.telegram_id}.txt"
    await callback.message.answer_document(
        buf,
        caption=f"■ ■■■ ■■■■■■■: {get_display_name(u)}"
    )
    await callback.answer()

# --- GIVE COINS ---
@router.callback_query(F.data == "admin_give_coins")
async def admin_give_coins_start(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    await state.set_state(AdminUserState.waiting_user_id)
    await state.update_data(action="give_coins")
    await callback.message.answer("■■■■■■ TG ID / @username / ■■■: ")
    await callback.answer()

@router.callback_query(F.data.startswith("give_coins_to:"))
async def give_coins_to_user(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    user_id = int(callback.data.split(":")[1])
    await state.set_state(AdminUserState.waiting_coins_amount)
    await state.update_data(target_user_id=user_id)
    await callback.message.answer(
        "■■■■■■ ■■■■■■■ ■■■ (■■■■■■■■, -■■■■■■): "
    )
    await callback.answer()

@router.message(AdminUserState.waiting_user_id)
async def process_user_id_for_action(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    data = await state.get_data()
    action = data.get("action")
    query = message.text.strip()

    async with async_session() as session:
        user = None
        if query.startswith("@"):
            user = await get_user_by_username(session, query)
        elif query.isdigit():
            user = await get_user(session, int(query))
        else:
            user = await get_user_by_display_name(session, query)

    if not user:
        await message.answer("■ ■■■■■■■■■ ■■ ■■■■■.")
        await state.clear()
        return

    await state.update_data(
        target_user_id=user.id,
        target_tg_id=user.telegram_id
    )

    if action == "give_coins":
        await state.set_state(AdminUserState.waiting_coins_amount)
        await message.answer("■■■■■■ ■■■■■■■ ■■■: ")
    elif action == "ban":
        ok = await set_user_ban_status(
            session, user.id, True, message.from_user.id
        )
        await message.answer(
            f"■ {get_display_name(user)} ■■■■■■■■■. " if ok else "■ ■■■■■■■."
        )
        if ok:
            try:

```

```

        await message.bot.send_message(
            user.telegram_id, "■ ■■ ████████████████████."
        )
    except Exception:
        pass
    await state.clear()
elif action == "unban":
    ok = await set_user_ban_status(
        session, user.id, False, message.from_user.id
    )
    await message.answer(
        f"■ {get_display_name(user)} ████████████████████." if ok else "■ ████████."
    )
    if ok:
        try:
            await message.bot.send_message(
                user.telegram_id, "■ ■■ ████████████████████."
            )
        except Exception:
            pass
    await state.clear()
elif action == "message":
    await state.set_state(AdminUserState.waiting_message_text)
    await message.answer("██████████ ████████ ████████████████:")

@router.message(AdminUserState.waiting_coins_amount)
async def process_coins_amount(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    try:
        amount = Decimal(message.text.strip())
    except Exception:
        await message.answer("■ ██████████ ████████ (50 █████ -10).")
        return

    data = await state.get_data()
    user_id = data.get("target_user_id")
    if not user_id:
        await message.answer("■ ████████: ████████████████████ ■■ ████████.")
        await state.clear()
        return

    async with async_session() as session:
        ok = await update_user_balance(
            session, user_id, amount, message.from_user.id
        )
        if ok:
            user = await get_user_by_id(session, user_id)
            name = get_display_name(user) if user else str(user_id)
            await message.answer(
                f"■ ██████████ ████████████████!\n"
                f"■ {name}\n"
                f"██████████: {'+' if amount > 0 else ''}{amount}\n"
                f"██████████ ██████████: {user.balance if user else '???'}"
            )
            if user:
                try:
                    await message.bot.send_message(
                        user.telegram_id,
                        f"■ ■■ ██████████ ████████████████████: "
                        f"{'+' if amount > 0 else ''}{amount} ████████"
                    )
                except Exception:
                    pass
            else:
                await message.answer("■ ██████████ ■■ ████████████████.")
        await state.clear()

# --- BAN / UNBAN ---
@router.callback_query(F.data == "admin_ban_user")
async def admin_ban_start(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    await state.set_state(AdminUserState.waiting_user_id)
    await state.update_data(action="ban")
    await callback.message.answer("██████████ ID/████/username █████ ████████████████:")
    await callback.answer()

@router.callback_query(F.data == "admin_unban_user")
async def admin_unban_start(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    await state.set_state(AdminUserState.waiting_user_id)

```

```

await state.update_data(action="unban")
await callback.message.answer("████████ ID/███/@username ██████████:")
await callback.answer()

@router.callback_query(F.data.startswith("ban_user:"))
async def ban_user_direct(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    user_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        ok = await set_user_ban_status(
            session, user_id, True, callback.from_user.id
        )
    user = await get_user_by_id(session, user_id)
    if ok:
        await callback.answer("■ ██████████!", show_alert=True)
        if user:
            try:
                await callback.bot.send_message(
                    user.telegram_id, "■ ■ ██████████."
                )
            except Exception:
                pass
    else:
        await callback.answer("■ ██████.", show_alert=True)

@router.callback_query(F.data.startswith("unban_user:"))
async def unban_user_direct(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    user_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        ok = await set_user_ban_status(
            session, user_id, False, callback.from_user.id
        )
    user = await get_user_by_id(session, user_id)
    if ok:
        await callback.answer("■ ██████████!", show_alert=True)
        if user:
            try:
                await callback.bot.send_message(
                    user.telegram_id, "■ ■ ██████████."
                )
            except Exception:
                pass
    else:
        await callback.answer("■ ██████.", show_alert=True)

# --- MESSAGE USER ---
@router.callback_query(F.data == "admin_message_user")
async def admin_message_user_start(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    await state.set_state(AdminUserState.waiting_user_id)
    await state.update_data(action="message")
    await callback.message.answer("████████ ID/███/@username ██████████:")
    await callback.answer()

@router.message(AdminUserState.waiting_message_text)
async def process_message_text(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    data = await state.get_data()
    target_tg_id = data.get("target_tg_id")
    if not target_tg_id:
        await message.answer("■ ██████: ██████████ ■ ██████.")
        await state.clear()
        return
    try:
        await message.bot.send_message(
            target_tg_id,
            f"■ <b>██████████ ████████████████████: </b>\n\n{message.text}",
            parse_mode="HTML"
        )
        await message.answer("■ ██████████ ██████████!")
    except Exception as e:
        await message.answer(f"■ ██████ ██████████: {e}")
        await state.clear()

# --- NICKNAME ---

```

[illegible]

```
@router.message(AdminNicknameState.waiting_user_id)
async def admin_nick_process_user(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    query = message.text.strip()
    async with async_session() as session:
        user = None
        if query.startswith("@"):
            user = await get_user_by_username(session, query)
        elif query.isdigit():
            user = await get_user(session, int(query))
        else:
            user = await get_user_by_display_name(session, query)

    if not user:
        await message.answer("■ ■■■■■■■■■■■■ ■■ ■■■■■■.")
        await state.clear()
        return

    await state.update_data(target_user_id=user.id)
    await state.set_state(AdminNicknameState.waiting_new_nick)
    await message.answer(
        f"■■■■■■■ ■■■■■ ■■■ ■■■ {get_display_name(user)} (■■■ '■■■■■'):"
    )
```



```

        Offer.is_active == True,
        Offer.status == "approved"
    )
)).scalar_one()

kb = InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(
        text="■ ■■■■■■■■ ■■■■■■ (■■■■■■■■■■)",
        callback_data="admin_create_offer"
    )],
    [InlineKeyboardButton(
        text=f"■ ■■ ■■■■■■■■ ({pending_count})",
        callback_data="admin_offers_pending"
    )],
    [InlineKeyboardButton(
        text=f"■ ■■■ ■■■■■■■■ ({total_count})",
        callback_data="admin_offers_all"
    )],
    [InlineKeyboardButton(
        text=f"■ ■■■■■■■■ ({active_count})",
        callback_data="admin_offers_active"
    )],
    [InlineKeyboardButton(
        text="■ ■■■■■■■■■■ ■■■■■■■■",
        callback_data="admin_rentals_menu"
    )],
    [InlineKeyboardButton(text="■ ■■■■■■", callback_data="admin_center")],
])
await _safe_edit(
    callback,
    f"■ <b>■■■■■■■■■■ ■■■■■■■■</b>\n\n"
    f"■■■■■: {total_count} | ■■■■■■■■: {active_count} | "
    f"■■ ■■■■■■■■: {pending_count}",
    parse_mode="HTML",
    reply_markup=kb
)
await callback.answer()

@router.callback_query(F.data == "admin_create_offer")
async def admin_create_offer_start(callback: CallbackQuery, state: FSMContext):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    await state.set_state(AdminOfferCreateState.waiting_title)
    await callback.message.answer(
        "■ <b>■■■■■■■■■■ ■■■■■■■■ (■■■ 1/8)</b>\n\n"
        "■■■■■■■ ■■■■■■■■ ■■■■■■ (■■■■■■■■■ ■■■■■■/■■■■■■■):",
        parse_mode="HTML"
    )
    await callback.answer()

@router.message(AdminOfferCreateState.waiting_title)
async def admin_offer_title(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    if len(message.text) > 100:
        await message.answer("■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■. ■■■■. 100 ■■■■■■■■.")
        return
    await state.update_data(title=message.text.strip())
    await state.set_state(AdminOfferCreateState.waiting_description)
    await message.answer(
        "■ <b>■■■ 2/8</b>\n\n■■■■■■■■■ ■■■■■■■■ ■■■■■■■■:",
        parse_mode="HTML"
    )

@router.message(AdminOfferCreateState.waiting_description)
async def admin_offer_description(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    await state.update_data(description=message.text.strip())
    await state.set_state(AdminOfferCreateState.waiting_url)
    await message.answer(
        "■ <b>■■■ 3/8</b>\n\n■■■■■■■■■ ■■■■■■■■ ■■ ■■■■■■ (https://t.me/...):",
        parse_mode="HTML"
    )

@router.message(AdminOfferCreateState.waiting_url)
async def admin_offer_url(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    url = message.text.strip()
    if not (url.startswith("https://t.me/") or url.startswith("t.me/")):
        await message.answer(
            "■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■ ■ https://t.me/ ■■■■ t.me/"

```



```

    if val <= 0:
        raise ValueError
    except Exception:
        await message.answer("■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■.")
        return
    await state.update_data(rent_cost_per_day=val)
    await state.set_state(AdminOfferCreateState.waiting_max_rentals)
    await message.answer(
        "■ <b>■■■ 8/8</b>\n\n"
        "■■■■■■■■■■ ■■■■ ■■■■■■■■■■ ■■■■■■■■■■? \n"
        "■■■■■■■■■■: 1-5",
        parse_mode="HTML"
    )

@router.message(AdminOfferCreateState.waiting_max_rentals)
async def admin_offer_max_rentals(message: Message, state: FSMContext):
    if not await check_admin(message.from_user.id):
        return
    try:
        val = int(message.text.strip())
        if val < 1:
            raise ValueError
    except Exception:
        await message.answer("■ ■■■■■■■■ ■■■■ ■■■■ >= 1.")
        return
    await state.update_data(max_simultaneous_rentals=val)
    await _finish_offer_creation(message, state)

async def _finish_offer_creation(
    message: Message,
    state: FSMContext,
    skip_rentals: bool = False
):
    data = await state.get_data()
    async with async_session() as session:
        offer = await admin_create_offer(
            session,
            title=data["title"],
            description=data["description"],
            channel_url=data["url"],
            reward_preview=data["reward_preview"],
            reward_final=data["reward_final"],
            is_rentable=data.get("is_rentable", False),
            rent_cost_per_day=data.get("rent_cost_per_day", Decimal("0")),
            max_simultaneous_rentals=data.get("max_simultaneous_rentals", 0),
        )

    rentable_text = ""
    if data.get("is_rentable"):
        rentable_text = (
            f"\n■ ■■■■■■■■: {data.get('rent_cost_per_day')} ■■■■/■■■■\n"
            f"■■■■. ■■■■■■■■■■: {data.get('max_simultaneous_rentals')}"
        )

    await message.answer(
        f"■ <b>■■■■ ■■■■■■■■!</b>\n\n"
        f"■ {data['title']}\n"
        f"■ ■■■■. ■■■■■■■■: {data['reward_preview']}\n"
        f"■ ■■■■. ■■■■■■■■: {data['reward_final']}"
        f"{rentable_text}\n\n"
        f"■■■■ ■■■■ ■■■■■■■■ ■ ■■■■ ■■■■■■■■■■.",
        parse_mode="HTML"
    )
    await state.clear()

@router.callback_query(F.data == "admin_offers_pending")
async def admin_offers_pending(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        offers = (await session.execute(
            select(Offer).where(Offer.status == "pending")
            .order_by(Offer.created_at)
        )).scalars().all()

    if not offers:
        await callback.message.answer(
            "■ ■■■ ■■■■■■■■ ■■ ■■■■■■■■■■.",
            reply_markup=InlineKeyboardMarkup(inline_keyboard=[
                [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_offers_menu")]
            ])
        )
    await callback.answer()
    return

```

```

kb_buttons = []
for offer in offers:
    kb_buttons.append([InlineKeyboardButton(
        text=f"■ {offer.title[:35]}",
        callback_data=f"admin_review_offer:{offer.id}"
    )])
kb_buttons.append([InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_offers_menu")])
await callback.message.answer(
    f"■ <b>■■■■■ ■■ ■■■■■ ({{len(offers)}})</b>",
    parse_mode="HTML",
    reply_markup=InlineKeyboardMarkup(inline_keyboard=kb_buttons)
)
await callback.answer()

@router.callback_query(F.data == "admin_offers_all")
async def admin_offers_all(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        offers = (await session.execute(
            select(Offer).order_by(desc(Offer.created_at)).limit(25)
        )).scalars().all()

    if not offers:
        await callback.message.answer("■■■■■■ ■■■.")
        await callback.answer()
        return

    text = "■ <b>■■■ ■■■■■ (■■■■■■■■ 25)</b>\n\n"
    for o in offers:
        icon = "■" if o.is_active else ("■" if o.status == "pending" else "■")
        rent_icon = "■" if getattr(o, "is_rentable", False) else ""
        text += (
            f"{{icon}}{{rent_icon}} #{{o.id}} <b>{{o.title[:30]}}</b>\n"
            f"   ■■■■■: {{o.status}} | "
            f"■■■■■■: {{o.reward_preview}}+{{o.reward_final}}\n"
        )

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_offers_menu")]
    ])
    if len(text) > 4000:
        text = text[:4000] + "\n..."
    await callback.message.answer(text, parse_mode="HTML", reply_markup=kb)
    await callback.answer()

@router.callback_query(F.data == "admin_offers_active")
async def admin_offers_active(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        offers = (await session.execute(
            select(Offer).where(
                Offer.is_active == True,
                Offer.status == "approved"
            ).order_by(desc(Offer.created_at))
        )).scalars().all()

    if not offers:
        await callback.message.answer(
            "■■■ ■■■■■ ■■■■■.",
            reply_markup=InlineKeyboardMarkup(inline_keyboard=[
                [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_offers_menu")]
            ])
        )
        await callback.answer()
        return

    kb_buttons = []
    for o in offers:
        rent_icon = "■" if getattr(o, "is_rentable", False) else ""
        kb_buttons.append([InlineKeyboardButton(
            text=f"■{{rent_icon}} #{{o.id}} {{o.title[:30]}}",
            callback_data=f"admin_review_offer:{{o.id}}"
        )])
    kb_buttons.append([InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_offers_menu")])
    await callback.message.answer(
        f"■ <b>■■■■■■ ■■■■■ ({{len(offers)}})</b>",
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup(inline_keyboard=kb_buttons)
    )
    await callback.answer()

```

```
@router.callback_view(F.data.startswith("admin_review_offer:"))
async def admin_review_offer(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = (await session.execute(
            select(Offer).where(Offer.id == offer_id)
        )).scalar_one_or_none()
    if not offer:
        await callback.answer("❌ ❌❌❌❌.", show_alert=True)
        return

    participants = (await session.execute(
        select(func.count(OfferParticipation.id)).where(
            OfferParticipation.offer_id == offer_id
        )
    )).scalar_one()

    completed = (await session.execute(
        select(func.count(OfferParticipation.id)).where(
            OfferParticipation.offer_id == offer_id,
            OfferParticipation.status == "completed"
        )
    )).scalar_one()

    active_rentals_count = 0
    try:
        active_rentals_count = (await session.execute(
            select(func.count(OfferRental.id)).where(
                OfferRental.offer_id == offer_id,
                OfferRental.status == "active"
            )
        )).scalar_one()
    except Exception:
        pass

    is_rentable = getattr(offer, "is_rentable", False)
    rent_cost = getattr(offer, "rent_cost_per_day", 0)
    max_rentals = getattr(offer, "max_simultaneous_rentals", 0)

    text = (
        f"■ <b>■■■■■ #{offer.id}</b>\n\n"
        f"■■■■■■■: <b>{offer.title}</b>\n"
        f"■■■■■■■: {offer.description}\n"
        f"URL: {offer.channel_url}\n"
        f"■■■■■■■: <b>{offer.status}</b> | "
        f"■■■■■■■: {'■' if offer.is_active else '■'}\n"
        f"■ ■■■■■■: {offer.reward_preview} + {offer.reward_final}\n"
        f"■■■■■■■■■: {participants} | ■■■■■■■■: {completed}\n"
        f"■ ■■■■■■: {'■' if is_rentable else '■'}"
    )
    if is_rentable:
        text += (
            f"\n ■■■■: {rent_cost} ■■■■/■■■■\n"
            f" ■■■■. ■■■■■■■■: {max_rentals}\n"
            f" ■■■■■■ ■■■■: {active_rentals_count}"
        )
    text += f"\n■ ■■■■■■: {offer.created_at.strftime('%d.%m.%Y %H:%M')}"

    action_buttons = []
    if offer.status == "pending":
        action_buttons.append([
            InlineKeyboardButton(
                text="■ ■■■■■■■■",
                callback_data=f"approve_offer:{offer_id}"
            ),
            InlineKeyboardButton(
                text="■ ■■■■■■■■",
                callback_data=f"reject_offer:{offer_id}"
            ),
        ])
    else:
        toggle_text = "■ ■■■■■■■■■■■■■■■■" if offer.is_active else "■ ■■■■■■■■■■■■■■■■"
        action_buttons.append([
            InlineKeyboardButton(
                text=toggle_text,
                callback_data=f"toggle_offer:{offer_id}"
            ),
        ])
    ])
```

```

])
action_buttons.append([
    InlineKeyboardButton(text="■ ■■■■■■", callback_data="admin_offers_menu")
])

await callback.message.answer(
    text, parse_mode="HTML",
    reply_markup=InlineKeyboardMarkup(inline_keyboard=action_buttons)
)
await callback.answer()

@router.callback_query(F.data.startswith("approve_offer:"))
async def approve_offer_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = (await session.execute(
            select(Offer).where(Offer.id == offer_id)
        )).scalar_one_or_none()
    if not offer:
        await callback.answer("■ ■■■■■■■■.", show_alert=True)
        return
    offer.status = "approved"
    offer.is_active = True
    await session.commit()

    if offer.creator_user_id:
        creator = await get_user_by_id(session, offer.creator_user_id)
        if creator:
            try:
                await callback.bot.send_message(
                    creator.telegram_id,
                    f"■ ■■■ ■■■■■ «{offer.title}» ■■■■■■ ■ ■■■■■■■■■■■!"
                )
            except Exception:
                pass
        await callback.answer("■ ■■■■■■ ■■■■■■■■■!", show_alert=True)

@router.callback_query(F.data.startswith("reject_offer:"))
async def reject_offer_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = (await session.execute(
            select(Offer).where(Offer.id == offer_id)
        )).scalar_one_or_none()
    if not offer:
        await callback.answer("■ ■■■■■■■■.", show_alert=True)
        return
    offer.status = "rejected"
    offer.is_active = False
    await session.commit()

    if offer.creator_user_id:
        creator = await get_user_by_id(session, offer.creator_user_id)
        if creator:
            try:
                await callback.bot.send_message(
                    creator.telegram_id,
                    f"■ ■■■ ■■■■■ «{offer.title}» ■■■■■■■■■."
                )
            except Exception:
                pass
        await callback.answer("■ ■■■■■■ ■■■■■■■■■!", show_alert=True)

@router.callback_query(F.data.startswith("toggle_offer:"))
async def toggle_offer_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = (await session.execute(
            select(Offer).where(Offer.id == offer_id)
        )).scalar_one_or_none()
    if not offer:
        await callback.answer("■ ■■■■■■■■.", show_alert=True)
        return
    offer.is_active = not offer.is_active
    await session.commit()
    status = "■■■■■■■■■■■ ■" if offer.is_active else "■■■■■■■■■■■■■■■■ ■"

```

```

await callback.answer(f"██████ {status}!", show_alert=True)

@router.callback_query(F.data.startswith("delete_offer:"))
async def delete_offer_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    offer_id = int(callback.data.split(":")[1])
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(
                text="■ ███, ████████",
                callback_data=f"confirm_delete_offer:{offer_id}"
            ),
            InlineKeyboardButton(
                text="■ ████████",
                callback_data=f"admin_review_offer:{offer_id}"
            ),
        ]
    ])
    await callback.message.answer(
        f"██ ████████ ██████ #{offer_id}? \n"
        f"██ ████████ ██████ ██████ ███ ████████████████████.",
        reply_markup=kb
    )
    await callback.answer()

@router.callback_query(F.data.startswith("confirm_delete_offer:"))
async def confirm_delete_offer_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = (await session.execute(
            select(Offer).where(Offer.id == offer_id)
        )).scalar_one_or_none()
        if not offer:
            await callback.answer("██ ████████.", show_alert=True)
            return
        offer.is_active = False
        offer.status = "deleted"
        await session.commit()
    await callback.answer(f"██ ████████ #{offer_id} ████████.", show_alert=True)

# =====
# RENTALS MANAGEMENT
# =====
@router.callback_query(F.data == "admin_rentals_menu")
async def admin_rentals_menu(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        try:
            active_count = await count_active_rentals(session)
            recent_rentals = (await session.execute(
                select(OfferRental, User, Offer)
                .join(User, User.id == OfferRental.renter_user_id)
                .join(Offer, Offer.id == OfferRental.offer_id)
                .order_by(desc(OfferRental.created_at))
                .limit(10)
            )).all()
        except Exception:
            active_count = 0
            recent_rentals = []

    text = f"██ <b>████████████████████ ██████████</b>\n\n████████████████████ ████████: {active_count}\n\n"

    kb_buttons = []
    if recent_rentals:
        text += "<b>████████████████████ ██████████</b>\n\n"
        for rental, user, offer in recent_rentals:
            expires = rental.expires_at.strftime('%d.%m') if rental.expires_at else "???"
            text += (
                f"• {get_display_name(user)} → {offer.title[:20]}\n"
                f"  {rental.renter_channel_title[:25]} | "
                f"██ {expires} | {rental.status}\n"
            )
            if rental.status == "pending":
                kb_buttons.append([InlineKeyboardButton(
                    text=f"██ ████████████████████: {rental.renter_channel_title[:25]}",
                    callback_data=f"admin_review_rental:{rental.id}"
                )])

```

```

kb_buttons.extend([
    [InlineKeyboardButton(
        text="■ ■■■■■■■■ ■■■■■■■■■■",
        callback_data="admin_expire_rentals"
    )],
    [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_offers_menu")],
])

if len(text) > 4000:
    text = text[:4000] + "\n..."

await callback.message.answer(
    text, parse_mode="HTML",
    reply_markup=InlineKeyboardMarkup(inline_keyboard=kb_buttons)
)
await callback.answer()

@router.callback_query(F.data.startswith("admin_review_rental:"))
async def admin_review_rental(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    rental_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        try:
            rental = (await session.execute(
                select(OfferRental).where(OfferRental.id == rental_id)
            )).scalar_one_or_none()
        except Exception:
            rental = None

    if not rental:
        await callback.answer("■■■■■■ ■■ ■■■■■■■■.", show_alert=True)
        return

    renter = await get_user_by_id(session, rental.renter_user_id)
    offer = (await session.execute(
        select(Offer).where(Offer.id == rental.offer_id)
    )).scalar_one_or_none()

    text = (
        f"■ <b>■■■■■■■ #{rental.id}</b>\n\n"
        f"■■■■■■■■■: {get_display_name(renter) if renter else '???'}\n"
        f"■■■■■■■: {rental.renter_channel_title}\n"
        f"■■■■■■■: {rental.renter_channel_url}\n"
        f"■■■■■■■: #{rental.offer_id} {offer.title if offer else '???'}\n"
        f"■■■■■: {rental.rent_days}\n"
        f"■■■■■■■■■■: {rental.cost_paid} ■■■■■\n"
        f"■■■■■■■: {rental.status}\n"
        f"■■■■■■■■■: {rental.created_at.strftime('%d.%m.%Y %H:%M')}\n"
        f"■■■■■■■■■: {rental.expires_at.strftime('%d.%m.%Y')} if rental.expires_at else '???'})
    )

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(
                text="■ ■■■■■■■■",
                callback_data=f"approve_rental:{rental_id}"
            ),
            InlineKeyboardButton(
                text="■ ■■■■■■■■",
                callback_data=f"reject_rental:{rental_id}"
            ),
        ],
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_rentals_menu")],
    ])
    await callback.message.answer(text, parse_mode="HTML", reply_markup=kb)
    await callback.answer()

@router.callback_query(F.data.startswith("approve_rental:"))
async def approve_rental_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    rental_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        try:
            rental = (await session.execute(
                select(OfferRental).where(OfferRental.id == rental_id)
            )).scalar_one_or_none()
        except Exception:
            rental = None
        if not rental:
            await callback.answer("■■ ■■■■■■■■.", show_alert=True)
            return
        rental.status = "active"
        rental.expires_at = datetime.utcnow() + timedelta(days=rental.rent_days)
        await session.commit()

```

```

        renter = await get_user_by_id(session, rental.renter_user_id)
        if renter:
            try:
                await callback.bot.send_message(
                    renter.telegram_id,
                    f"■ ■■■■ ■■■■■■ ■■■■■■■■■■!\n"
                    f"■■■■■: {rental.renter_channel_title}\n"
                    f"■■■■■■■ ■■: {rental.expires_at.strftime('%d.%m.%Y')}}"
                )
            except Exception:
                pass
        except Exception as e:
            await callback.answer(f"■■■■■■■: {e}", show_alert=True)
        return
    await callback.answer("■ ■■■■■■ ■■■■■■■■■■!", show_alert=True)

@router.callback_query(F.data.startswith("reject_rental:"))
async def reject_rental_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    rental_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        try:
            rental = (await session.execute(
                select(OfferRental).where(OfferRental.id == rental_id)
            )).scalar_one_or_none()
            if not rental:
                await callback.answer("■■ ■■■■■■■■.", show_alert=True)
                return

            rental.status = "rejected"
            renter = await get_user_by_id(session, rental.renter_user_id)
            if renter and rental.cost_paid > 0:
                from app.services import log_balance_change
                await log_balance_change(
                    session, renter, rental.cost_paid,
                    "rental_refund", source_id=rental_id,
                    details="■■■■■■■ ■■ ■■■■■■■■■■ ■■■■■■"
                )
            renter.balance += rental.cost_paid
            await session.commit()

            if renter:
                try:
                    await callback.bot.send_message(
                        renter.telegram_id,
                        f"■ ■■■■ ■■■■■■ ■■■■■■■■■■.\n"
                        f"■■■■■■■: {rental.cost_paid} ■■■■■■"
                    )
                except Exception:
                    pass
        except Exception as e:
            await callback.answer(f"■■■■■■■: {e}", show_alert=True)
        return
    await callback.answer("■ ■■■■■■ ■■■■■■■■■■, ■■■■■■ ■■■■■■■■■■.", show_alert=True)

@router.callback_query(F.data == "admin_expire_rentals")
async def admin_expire_rentals_cb(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        await callback.answer()
        return
    async with async_session() as session:
        try:
            expired = await expire_old_rentals(session)
            await callback.answer(f"■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■: {expired}", show_alert=True)
        except Exception as e:
            await callback.answer(f"■ ■■■■■■: {e}", show_alert=True)

# =====
# ADMIN MANAGEMENT
# =====
@router.callback_query(F.data == "admin_manage_admins")
async def manage_admins_menu(callback: CallbackQuery):
    if not is_super_admin(callback.from_user.id):
        await callback.answer()
        return
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■", callback_data="admin_list_admins")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_add_admin")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_remove_admin")],
        [InlineKeyboardButton(text="■ ■■■■■■", callback_data="admin_center")],
    ])
    await _safe_edit(

```


[illegible]

[illegible]

```

VIP_FREE_PROMO_PER_MONTH = 1
# =====
#
# =====
DICE_MIN_BET = _get_int("DICE_MIN_BET", 1)
DICE_MAX_BET = _get_int("DICE_MAX_BET", 50)

# =====
#
# =====
DAILY_QUESTS = [
    {"type": "watch", "target": 3, "reward": 2, "desc": "3"},
    {"type": "upload", "target": 1, "reward": 3, "desc": "1"},
    {"type": "rate", "target": 3, "reward": 1, "desc": "3"},
    {"type": "comment", "target": 2, "reward": 2, "desc": "2"},
    {"type": "react", "target": 5, "reward": 1, "desc": "5"},
]
PREMIUM_DAILY_QUESTS = [
    {"type": "watch", "target": 10, "reward": 8, "desc": "VIP: 10"},
    {"type": "comment", "target": 5, "reward": 5, "desc": "VIP: 5"},
]

REACTION_TYPES = ["", "❤️", "👍", "👎", "👁️"]

# =====
#
# =====
COMMENTS_PER_10_MIN = _get_int("COMMENTS_PER_10_MIN", 5)
COMMENT_MIN_INTERVAL_SEC = _get_int("COMMENT_MIN_INTERVAL_SEC", 15)

# =====
#
# =====
WEEKLY_TOP1_REWARD = _get_float("WEEKLY_TOP1_REWARD", 50.0)
WEEKLY_TOP2_REWARD = _get_float("WEEKLY_TOP2_REWARD", 25.0)
WEEKLY_TOP3_REWARD = _get_float("WEEKLY_TOP3_REWARD", 15.0)

# =====
#
# =====
PIN_OFFER_COST = _get_float("PIN_OFFER_COST", 100.0)
BUMP_VIDEO_COST = _get_float("BUMP_VIDEO_COST", 25.0)

# =====
#
# =====
NICKNAME_FIRST_FREE = True
NICKNAME_CHANGE_COST = _get_float("NICKNAME_CHANGE_COST", 50.0)
NICKNAME_MIN_LENGTH = 3
NICKNAME_MAX_LENGTH = 20

# =====
#
# =====
OFFER_DEFAULT_RENT_COST_PER_DAY = _get_float("OFFER_DEFAULT_RENT_COST_PER_DAY", 5.0)
OFFER_MIN_RENT_DAYS = 1
OFFER_MAX_RENT_DAYS = 30

# =====
#
# =====
PROMOCODE_CREATOR_BONUS_PERCENT = 5.0

# =====
#
# =====
PROMOCODE_MAX_AMOUNT = 1000
PROMOCODE_MAX_USES = 100
PROMOCODE_MAX_HOURS = 168

# =====
#
# =====
DYNAMIC_STAR_DISCOUNT_ENABLED = True
DYNAMIC_STAR_DISCOUNT_HOURS = "17-20"
DYNAMIC_STAR_DISCOUNT_MULTIPLIER = 1.5
FIRST_PURCHASE_DAILY_BONUS = 5.0

```

```

# ■■■■■ ■■■■■ ■■■■■
# =====
SMART_AD_MIN_INTERVAL_MINUTES = 15
SMART_AD_LOW_BALANCE_THRESHOLD = 3.0
SMART_AD_LOW_BALANCE_HINT_INTERVAL_MINUTES = 30
SMART_AD_VIDEO_CHANCE = 0.35
SMART_AD_FORCED_WATCH_SECONDS = 5

=====
FILE: app\db.py
=====
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession

from app.config import DATABASE_URL
from app.models import Base

def _fix_url(url: str) -> str:
    if url.startswith("postgres://"):
        return url.replace("postgres://", "postgresql+asyncpg://", 1)
    if url.startswith("postgresql://"):
        return url.replace("postgresql://", "postgresql+asyncpg://", 1)
    return url

engine = create_async_engine(
    _fix_url(DATABASE_URL),
    echo=False,
)

async_session = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

async def reset_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
        await conn.run_sync(Base.metadata.create_all)

=====
FILE: app\keyboards.py
=====
from aiogram.types import (
    InlineKeyboardMarkup, InlineKeyboardButton,
    KeyboardButton, ReplyKeyboardMarkup
)

# =====
# ■■■■■ ■■■■■ ■■■■■ ■■■■■
# =====
BTN_WATCH      = "■ ■■■■■■■■"
BTN_UPLOAD     = "■ ■■■■■■■■"
BTN_PROFILE    = "■ ■■■■■■■■"
BTN_BUY        = "■ ■■■■■■ ■■■■■■"
BTN_OFFERS     = "■ ■■■■■■"
BTN_REFERRALS  = "■ ■■■■■■■■"
BTN_BONUS      = "■ ■■■■■■"
BTN_ADMIN      = "■ ■■■■■■■■"
BTN_GAMES      = "■ ■■■■"
BTN_TOPS       = "■ ■■■■"
BTN_QUESTS     = "■ ■■■■■■■■"
BTN_VIP        = "■ VIP"
BTN_LEVEL      = "■ ■■■■■■■■"
BTN_PROMO      = "■ ■■■■■■■■"

# =====
# ■■■■■ ■■■■ (ReplyKeyboard)
# =====
def main_menu(is_admin: bool = False) -> ReplyKeyboardMarkup:
    kb = [
        [KeyboardButton(text=BTN_WATCH)],
        [KeyboardButton(text=BTN_UPLOAD), KeyboardButton(text=BTN_PROFILE)],
        [KeyboardButton(text=BTN_BUY), KeyboardButton(text=BTN_PROMO)],
        [KeyboardButton(text=BTN_OFFERS), KeyboardButton(text=BTN_REFERRALS)],
        [KeyboardButton(text=BTN_GAMES), KeyboardButton(text=BTN_BONUS)],
        [KeyboardButton(text=BTN_QUESTS), KeyboardButton(text=BTN_TOPS)],
        [KeyboardButton(text=BTN_VIP), KeyboardButton(text=BTN_LEVEL)],
    ]
    if is_admin:

```

```

        kb.append([KeyboardButton(text=BTN_ADMIN)])
    return ReplyKeyboardMarkup(keyboard=kb, resize_keyboard=True)

# =====
# ██████████-██████████████ (Inline)
# =====
def admin_main_keyboard(is_super: bool = False) -> InlineKeyboardMarkup:
    buttons = [
        [InlineKeyboardButton(text="■ ██████████ ██████████", callback_data="admin_queue_info"),
         [InlineKeyboardButton(text="■ ████████████████████████████████", callback_data="admin_extended_stats")],
         [InlineKeyboardButton(text="■ ████████████████████████████████", callback_data="admin_manage_users")],
         [InlineKeyboardButton(text="■ ████████ ████████████████████████████████", callback_data="admin_get_pending")],
         [InlineKeyboardButton(text="■ ██████████ █████", callback_data="admin_approve_all")],
         [InlineKeyboardButton(text="■ ██████████", callback_data="admin_offers_menu")],
         [InlineKeyboardButton(text="■ ████████████████████████████████", callback_data="admin_investigation")],
        ]
    if is_super:
        buttons.append([
            InlineKeyboardButton(
                text="■ █████ ██████████",
                callback_data="admin_db_menu"
            )
        ])
        buttons.append([
            InlineKeyboardButton(
                text="■ ████████████████████████████████",
                callback_data="admin_manage_admins"
            )
        ])
    return InlineKeyboardMarkup(inline_keyboard=buttons)

def admin_after_action_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ██████████", callback_data="admin_get_pending")],
        [InlineKeyboardButton(text="■ ██████████-████████", callback_data="admin_center")],
    ])

# =====
# ████████████████████████████████
# =====
def moderation_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="■ ██████████", callback_data=f"mod_approve:{video_id}"),
            InlineKeyboardButton(text="■ ██████████", callback_data=f"mod_reject:{video_id}"),
        ],
        [InlineKeyboardButton(text="■ ██████████", callback_data="admin_get_pending")],
    ])

def rejection_reason_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ██████████", callback_data=f"reject_reason:{video_id}:duplicate")],
        [InlineKeyboardButton(text="■ ██████████", callback_data=f"reject_reason:{video_id}:off_topic")],
        [InlineKeyboardButton(text="■ ██████████", callback_data=f"reject_reason:{video_id}:forbidden")],
        [InlineKeyboardButton(text="■ ██████████", callback_data=f"reject_reason:{video_id}:other")],
    ])

# =====
# ████████████████████████████████ ████████████████████████████████ (Inline)
# =====
def rules_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ██████████ ██████████", callback_data="accept_rules")],
    ])

def watch_choice_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ██████████", callback_data="watch_video_content")],
        [InlineKeyboardButton(text="■ ██████████", callback_data="watch_photo_content")],
    ])

def video_rating_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="1", callback_data=f"rate:{video_id}:1"),
            InlineKeyboardButton(text="2", callback_data=f"rate:{video_id}:2"),
            InlineKeyboardButton(text="3", callback_data=f"rate:{video_id}:3"),
            InlineKeyboardButton(text="4", callback_data=f"rate:{video_id}:4"),
            InlineKeyboardButton(text="5", callback_data=f"rate:{video_id}:5"),
        ],
        [InlineKeyboardButton(text="■ ████████████████████████████████", callback_data=f"comments:{video_id}")],
    ])

```

```

        [InlineKeyboardButton(text="■ ■■■■■■", callback_data=f"reactions:{video_id}")),
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="watch_next")],
    ])

def photo_actions_keyboard(photo_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■", callback_data=f"reactions:{photo_id}")),
        [InlineKeyboardButton(text="■ ■■■■■■■■ ■■■■", callback_data="watch_next_photo")],
    ])

def buy_coins_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ 2 ■■■■■■ (1 Star)", callback_data="buy:pack_1")),
        [InlineKeyboardButton(text="■ 10 ■■■■■■ (5 Stars)", callback_data="buy:pack_5")),
        [InlineKeyboardButton(text="■ 20 ■■■■■■ (10 Stars)", callback_data="buy:pack_10")),
        [InlineKeyboardButton(text="■ 50 ■■■■■■ (25 Stars)", callback_data="buy:pack_25")),
        [InlineKeyboardButton(text="■ 100 ■■■■■■ (50 Stars)", callback_data="buy:pack_50")),
        [InlineKeyboardButton(text="■ ■■■■ ■■■■■ Stars", callback_data="buy_custom")],
    ])

def vip_buy_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■ VIP", callback_data="buy_vip")],
    ])

# =====
# ■■■■■■
# =====
def offers_list_keyboard(offers: list) -> InlineKeyboardMarkup:
    buttons = []
    for o in offers:
        icon = "■" if o.is_active else "■"
        rent_icon = "■" if getattr(o, "is_rentable", False) else ""
        buttons.append([InlineKeyboardButton(
            text=f"{icon}{rent_icon} {o.title[:35]}",
            callback_data=f"offer_open:{o.id}"
        )])
    return InlineKeyboardMarkup(inline_keyboard=buttons)

def offer_view_keyboard(offer_id: int, channel_url: str) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■ ■ ■■■■■", url=channel_url)],
        [InlineKeyboardButton(text="■ ■■■■■■", callback_data=f"offer_start:{offer_id}")),
        [InlineKeyboardButton(text="■ ■■■■■■■■ ■■■■■■■■", callback_data=f"offer_check:{offer_id}")),
        [InlineKeyboardButton(text="■ ■■■■■■■■■■ ■■■■", callback_data=f"rent_offer:{offer_id}")),
    ])

def offer_rent_keyboard(offer_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■■■■■ ■■■■", callback_data=f"rent_offer:{offer_id}")),
        [InlineKeyboardButton(text="■ ■ ■■■■■■■■", callback_data="back_to_offers")],
    ])

def rent_days_keyboard(offer_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="1 ■■■■", callback_data=f"rent_days:{offer_id}:1"),
            InlineKeyboardButton(text="3 ■■■", callback_data=f"rent_days:{offer_id}:3"),
        ],
        [
            InlineKeyboardButton(text="7 ■■■■", callback_data=f"rent_days:{offer_id}:7"),
            InlineKeyboardButton(text="14 ■■■■", callback_data=f"rent_days:{offer_id}:14"),
        ],
        [
            InlineKeyboardButton(text="30 ■■■■", callback_data=f"rent_days:{offer_id}:30"),
            [InlineKeyboardButton(text="■ ■■■■■", callback_data=f"offer_open:{offer_id}")),
        ],
    ])

# =====
# ■■■■
# =====
def games_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="game_dice"),
        [InlineKeyboardButton(text="■ ■■■■/■■■■", callback_data="game_coinflip"),
        [InlineKeyboardButton(text="■ ■■■■■■ ■■■■■", callback_data="game_guess")],
    ])

# =====
# ■■■■

```

```

# =====
def tops_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■ ■■■■■■■■■■■■", callback_data="top_uploaders")],
        [InlineKeyboardButton(text="■ ■■■ ■■■■■■■■■■■■", callback_data="top_viewers")],
        [InlineKeyboardButton(text="■ ■■■ ■■ XP", callback_data="top_levels")],
        [InlineKeyboardButton(text="■ ■■■ ■■■■■■■■■■", callback_data="top_richest")],
    ])

# =====
# ■■■■■■
# =====
def quests_keyboard(quests: list) -> InlineKeyboardMarkup:
    buttons = []
    for q in quests:
        if q.completed and not q.reward_claimed:
            buttons.append([InlineKeyboardButton(
                text=f"■ ■■■■■■■■ ■■■■■■■■: {q.quest_type} ({q.reward} ■■■■■)",
                callback_data=f"quest_claim:{q.id}"
            )])
        elif q.completed:
            buttons.append([InlineKeyboardButton(
                text=f"■ {q.quest_type}: {q.progress}/{q.target} - ■■■■■■■■■■",
                callback_data=f"quest_done:{q.id}"
            )])
        else:
            buttons.append([InlineKeyboardButton(
                text=f"■ {q.quest_type}: {q.progress}/{q.target}",
                callback_data=f"quest_info:{q.id}"
            )])
    return InlineKeyboardMarkup(inline_keyboard=buttons)

def reaction_menu_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
            InlineKeyboardButton(text="♥", callback_data=f"react:{video_id}:♥"),
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
        ]
    ])

# =====
# ■■■■■■ ■■■■■■■■ (forced offer / low balance)
# =====
def forced_offer_keyboard(offer_id: int, channel_url: str, seconds: int = 5) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text=f"■ ■■■■■■■■ ■ ■■■■■■ (■■■■■■■■)",
            url=channel_url
        )],
        [InlineKeyboardButton(
            text=f"■ ■■■■■ {seconds} ■■■..",
            callback_data="forced_offer_wait"
        )],
    ])

def forced_offer_done_keyboard(offer_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text="■ ■■■■■■■■■■■■",
            callback_data=f"forced_offer_continue:{offer_id}"
        )],
    ])

def low_balance_offer_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text="■ ■■■■■■■■ ■ ■■■■■■■■",
            callback_data="offers_participation"
        )],
        [InlineKeyboardButton(
            text="■ ■■■■■■■■",
            callback_data="dismiss_low_balance_hint"
        )],
    ])

# =====
# ■■■■■■■■■■ ■■ (■■■■■■■■ ■■■■ super-admin)
# =====
def admin_db_keyboard(tables: list[str]) -> InlineKeyboardMarkup:

```



```

kb = []
for t in tables:
    kb.append([InlineKeyboardButton(text=f"■ {t}", callback_data=f"db_table:{t}"))
kb.append([InlineKeyboardButton(text="■ ■■■■■■-■■■■■", callback_data="admin_center")])
return InlineKeyboardMarkup(inline_keyboard=kb)

```

```

=====
FILE: app\logger.py
=====

```

```

import json
import logging
import traceback
from datetime import datetime

```

```

def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)

```

```

def _make_payload(message: str, **kwargs) -> str:
    payload = {
        "time": datetime.utcnow().isoformat() + "Z",
        "message": message,
    }
    payload.update(kwargs)
    return json.dumps(payload, ensure_ascii=False)

```

```

def setup_logging(level: int = logging.INFO):
    root = logging.getLogger()
    root.setLevel(level)

    while root.handlers:
        root.handlers.pop()

    handler = logging.StreamHandler()
    handler.setLevel(level)
    handler.setFormatter(logging.Formatter("%(message)s"))
    root.addHandler(handler)

```

```

def log_info(logger: logging.Logger, message: str, **kwargs):
    logger.info(_make_payload(message, **kwargs))

```

```

def log_warning(logger: logging.Logger, message: str, **kwargs):
    logger.warning(_make_payload(message, **kwargs))

```

```

def log_error(logger: logging.Logger, message: str, **kwargs):
    logger.error(_make_payload(message, **kwargs))

```

```

def log_exception(logger: logging.Logger, message: str, **kwargs):
    kwargs["traceback"] = traceback.format_exc()
    logger.error(_make_payload(message, **kwargs))

```

```

def log_event(logger: logging.Logger, level: int, message: str, **kwargs):
    logger.log(level, _make_payload(message, **kwargs))

```

```

=====
FILE: app\main.py
=====

```

```

import asyncio
from aiohttp import web
from aiogram import Bot, Dispatcher
from aiogram.client.default import DefaultBotProperties
from aiogram.enums import ParseMode

```

```

from app.config import BOT_TOKEN, PORT
from app.db import engine, init_db
from app.user_handlers import router as user_router
from app.admin_handlers import router as admin_router
from app.logger import setup_logging, get_logger, log_info

```

```

setup_logging()
logger = get_logger(__name__)

```

```

async def on_startup(app):
    from app.migrate import main as run_migrations
    await init_db()
    try:
        await run_migrations()
    except Exception as e:
        log_info(logger, f"Migrations warning: {e}")
    print("=" * 40)

```

```

print(" BOT STARTED")
print("=" * 40)
log_info(logger, "Bot started, DB initialized")

async def main():
    if not BOT_TOKEN:
        raise RuntimeError("BOT_TOKEN is empty")

    bot = Bot(
        token=BOT_TOKEN,
        default=DefaultBotProperties(parse_mode=ParseMode.HTML),
    )
    dp = Dispatcher()
    dp.include_router(user_router)
    dp.include_router(admin_router)

    app = web.Application()
    app.on_startup.append(on_startup)
    runner = web.AppRunner(app)
    await runner.setup()
    site = web.TCPSite(runner, "0.0.0.0", int(PORT or 10000))
    await site.start()

    try:
        log_info(logger, "Polling started")
        await dp.start_polling(bot)
    finally:
        await runner.cleanup()
        await bot.session.close()
        await engine.dispose()

if __name__ == "__main__":
    asyncio.run(main())

=====
FILE: app\make_project_pdf.py
=====
import os
import logging
from pathlib import Path
from datetime import datetime

from reportlab.lib.pagesizes import A4
from reportlab.pdfbase.pdfmetrics import stringWidth
from reportlab.pdfgen import canvas

import sys
sys.path.insert(0, str(Path(__file__).parent.parent))
from app.logger import setup_logging, get_logger, log_info, log_warning, log_exception
from pathlib import Path

PROJECT_ROOT = Path(".")
OUTPUT_FILE = "project_dump.pdf"

MAX_PAGES = 90
MAX_SIZE_MB = 15

INCLUDE_EXTENSIONS = {
    ".py",
    ".txt",
    ".md",
    ".json",
    ".yaml",
    ".yml",
    ".toml",
    ".ini",
    ".env",
    ".sql",
}

SPECIAL_FILENAMES = {
    "Dockerfile",
    "Procfile",
    "requirements.txt",
    "runtime.txt",
    ".env",
}

EXCLUDED_DIRS = {
    ".git",
    ".venv",
    "venv",
    "__pycache__",
    "node_modules",
    ".idea",
}

```

```

        ".vscode",
        "dist",
        "build",
        ".pytest_cache",
        ".mypy_cache",
        ".ruff_cache",
        ".cache",
    }

EXCLUDED_FILES = {
    OUTPUT_FILE,
}

MAX_FILE_SIZE_KB = 300

def is_text_candidate(path: Path) -> bool:
    if path.name in SPECIAL_FILENAMES:
        return True
    if path.suffix.lower() in INCLUDE_EXTENSIONS:
        return True
    if path.name.startswith(".env"):
        return True
    return False

def should_skip(path: Path) -> bool:
    if path.is_dir():
        return True

    if path.name in EXCLUDED_FILES:
        return True

    parts = set(path.parts)
    if parts & EXCLUDED_DIRS:
        return True

    if not is_text_candidate(path):
        return True

    try:
        if path.stat().st_size > MAX_FILE_SIZE_KB * 1024:
            return True
    except Exception:
        return True

    return False

def iter_project_files(root: Path):
    files = []
    for path in root.rglob("*"):
        if should_skip(path):
            continue
        files.append(path)
    files.sort(key=lambda p: str(p).lower())
    return files

def safe_read_text(path: Path) -> str:
    for enc in ("utf-8", "utf-8-sig", "cp1251", "latin-1"):
        try:
            return path.read_text(encoding=enc)
        except Exception:
            pass
    return "[[FAILED TO READ FILE]]"

def wrap_line(line: str, font_name: str, font_size: int, max_width: float):
    if not line:
        return [""]

    if stringWidth(line, font_name, font_size) <= max_width:
        return [line]

    parts = []
    current = ""

    for ch in line:
        test = current + ch
        if stringWidth(test, font_name, font_size) <= max_width:
            current = test
        else:
            if current:
                parts.append(current)
            current = ch

    if current:

```

```

        parts.append(current)

    return parts

def build_blocks(root: Path):
    files = iter_project_files(root)
    blocks = []

    header = [
        "<PROJECT PDF DUMP>",
        f"Generated at: {datetime.utcnow().isoformat()} UTC",
        f"Root: {root.resolve()}",
        f"Files included: {len(files)}",
        ""
    ]
    blocks.append("\n".join(header))

    for path in files:
        rel = path.relative_to(root)
        content = safe_read_text(path).rstrip()

        block = [
            "=" * 80,
            f"FILE: {rel}",
            "=" * 80,
            content,
            ""
        ]
        blocks.append("\n".join(block))

    return blocks, files

def render_pdf(blocks, output_path: str):
    page_width, page_height = A4

    configs = [
        {"font_size": 9, "line_gap": 11, "margin": 36},
        {"font_size": 8, "line_gap": 9, "margin": 26},
        {"font_size": 7, "line_gap": 8, "margin": 18},
        {"font_size": 6, "line_gap": 7, "margin": 14},
    ]

    best_result = None

    for cfg in configs:
        c = canvas.Canvas(output_path, pagesize=A4)
        font_name = "Courier"
        font_size = cfg["font_size"]
        line_gap = cfg["line_gap"]
        margin = cfg["margin"]

        usable_width = page_width - margin * 2
        y = page_height - margin
        pages = 1

        c.setFont(font_name, font_size)

        def new_page():
            nonlocal y, pages
            c.showPage()
            c.setFont(font_name, font_size)
            y = page_height - margin
            pages += 1

        for block in blocks:
            for raw_line in block.splitlines():
                wrapped = wrap_line(raw_line, font_name, font_size, usable_width)
                for line in wrapped:
                    if y < margin:
                        new_page()
                    c.drawString(margin, y, line)
                    y -= line_gap

            if y < margin:
                new_page()
            y -= line_gap

        c.save()

        size_bytes = os.path.getsize(output_path)
        size_mb = size_bytes / (1024 * 1024)

        best_result = {
            "pages": pages,
            "size_bytes": size_bytes,
            "size_mb": size_mb,

```

```

        "config": cfg,
    }

    if pages <= MAX_PAGES and size_mb <= MAX_SIZE_MB:
        return best_result

    return best_result

def main():
    setup_logging()
    logger = get_logger(__name__)

    try:
        blocks, files = build_blocks(PROJECT_ROOT)
        result = render_pdf(blocks, OUTPUT_FILE)

        logger.info("=" * 60)
        logger.info(f"PDF created: {OUTPUT_FILE}")
        logger.info(f"Files included: {len(files)}")
        logger.info(f"Pages: {result['pages']}")
        logger.info(f"Size: {result['size_mb']:.2f} MB")
        logger.info(f"Config used: {result['config']}")
        logger.info("=" * 60)

        if result["pages"] > MAX_PAGES:
            logger.warning(f"PDF exceeds page limit ({result['pages']} > {MAX_PAGES})")

        if result["size_mb"] > MAX_SIZE_MB:
            logger.warning(f"PDF exceeds size limit ({result['size_mb']:.2f} MB > {MAX_SIZE_MB} MB)")
    except Exception:
        log_exception(logger, "Error during PDF generation")

if __name__ == "__main__":
    main()

```

```

=====
FILE: app\migrate.py
=====
import asyncio
import sys
sys.path.insert(0, '..')

from sqlalchemy import text
from app.db import engine
from app.logger import setup_logging, get_logger, log_info

setup_logging()
logger = get_logger(__name__)

MIGRATIONS = [
    # --- offers ---
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS created_by_user_id INTEGER NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS offer_kind VARCHAR(30) DEFAULT 'system';
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promotion_tier VARCHAR(30) DEFAULT 'basic';
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS payment_method VARCHAR(20) NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_coins NUMERIC(12,2) DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_rub INTEGER DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_stars INTEGER DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS priority_level INTEGER DEFAULT 10;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS is_featured BOOLEAN DEFAULT FALSE;
    """
]

```

```

""",
""",
ALTER TABLE offers
ADD COLUMN IF NOT EXISTS moderation_comment TEXT NULL;
""",
""",
ALTER TABLE offers
ADD COLUMN IF NOT EXISTS approved_at TIMESTAMP NULL;
""",
""",
ALTER TABLE offers
ADD COLUMN IF NOT EXISTS is_rentable BOOLEAN DEFAULT FALSE;
""",
""",
ALTER TABLE offers
ADD COLUMN IF NOT EXISTS rent_cost_per_day NUMERIC(10,2) DEFAULT 0;
""",
""",
ALTER TABLE offers
ADD COLUMN IF NOT EXISTS max_simultaneous_rentals INTEGER DEFAULT 1;
""",
# --- user_ad_states ---
""",
CREATE TABLE IF NOT EXISTS user_ad_states (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL UNIQUE REFERENCES users(id),
    last_offer_shown_at TIMESTAMP NULL,
    last_low_balance_hint_at TIMESTAMP NULL,
    forced_offer_id INTEGER NULL,
    forced_offer_shown_at TIMESTAMP NULL,
    updated_at TIMESTAMP DEFAULT NOW()
);
""",
""",
CREATE INDEX IF NOT EXISTS ix_user_ad_states_user_id
ON user_ad_states (user_id);
""",
# --- users ---
""",
ALTER TABLE users
ADD COLUMN IF NOT EXISTS display_name VARCHAR(32) NULL;
""",
""",
ALTER TABLE users
ADD COLUMN IF NOT EXISTS nickname_set BOOLEAN DEFAULT FALSE;
""",
# --- game_sessions ---
""",
CREATE TABLE IF NOT EXISTS game_sessions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id),
    window_start TIMESTAMP NOT NULL,
    games_played INTEGER DEFAULT 0,
    paid_at TIMESTAMP NULL
);
""",
""",
CREATE INDEX IF NOT EXISTS ix_game_sessions_user_id
ON game_sessions (user_id);
""",
# --- balance_logs ---
""",
CREATE TABLE IF NOT EXISTS balance_logs (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id),
    amount NUMERIC(10,2) NOT NULL,
    balance_before NUMERIC(10,2) NOT NULL,
    balance_after NUMERIC(10,2) NOT NULL,
    source VARCHAR(100) NOT NULL,
    source_id INTEGER NULL,
    admin_id INTEGER NULL,
    details TEXT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);
""",
""",
CREATE INDEX IF NOT EXISTS ix_balance_logs_user_id
ON balance_logs (user_id);
""",
""",
CREATE INDEX IF NOT EXISTS ix_balance_logs_created_at
ON balance_logs (created_at);
""",
""",
CREATE INDEX IF NOT EXISTS ix_balance_logs_source
ON balance_logs (source);
""",
# --- offer_rentals ---

```

```

""
CREATE TABLE IF NOT EXISTS offer_rentals (
    id SERIAL PRIMARY KEY,
    offer_id INTEGER NOT NULL REFERENCES offers(id),
    renter_user_id INTEGER NOT NULL REFERENCES users(id),
    renter_channel_title VARCHAR(255) NOT NULL,
    renter_channel_url TEXT NOT NULL,
    rent_days INTEGER NOT NULL,
    cost_paid NUMERIC(10,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP NULL
);
""
""
CREATE INDEX IF NOT EXISTS ix_offer_rentals_offer_id
ON offer_rentals (offer_id);
""
""
CREATE INDEX IF NOT EXISTS ix_offer_rentals_renter_user_id
ON offer_rentals (renter_user_id);
""
""
CREATE INDEX IF NOT EXISTS ix_offer_rentals_status
ON offer_rentals (status);
""
# --- ████████ ---
""
CREATE INDEX IF NOT EXISTS ix_offers_created_by_user_id
ON offers (created_by_user_id);
""
""
CREATE INDEX IF NOT EXISTS ix_offers_status
ON offers (status);
""
""
CREATE INDEX IF NOT EXISTS ix_game_history_user_id
ON game_history (user_id);
""
""
CREATE INDEX IF NOT EXISTS ix_game_history_created_at
ON game_history (created_at);
""
""
ALTER TABLE users ADD COLUMN IF NOT EXISTS bonus_streak INTEGER DEFAULT 0;
""
""
ALTER TABLE users ADD COLUMN IF NOT EXISTS promo_created_this_month INTEGER DEFAULT 0;
""
""
CREATE TABLE IF NOT EXISTS promocodes (
    id SERIAL PRIMARY KEY,
    creator_user_id INTEGER NOT NULL REFERENCES users(id),
    code VARCHAR(50) UNIQUE NOT NULL,
    coin_amount NUMERIC(10,2) NOT NULL,
    max_uses INTEGER NOT NULL,
    used_count INTEGER DEFAULT 0,
    is_active BOOLEAN DEFAULT TRUE,
    is_public BOOLEAN DEFAULT FALSE,
    created_via_stars BOOLEAN DEFAULT TRUE,
    stars_paid INTEGER DEFAULT 0,
    expires_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT NOW()
);
""
""
CREATE INDEX IF NOT EXISTS ix_promocodes_code ON promocodes (code);
""
""
CREATE INDEX IF NOT EXISTS ix_promocodes_creator ON promocodes (creator_user_id);
""
""
CREATE TABLE IF NOT EXISTS promocode_activations (
    id SERIAL PRIMARY KEY,
    promocode_id INTEGER NOT NULL REFERENCES promocodes(id),
    user_id INTEGER NOT NULL REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW()
);
""
""
CREATE INDEX IF NOT EXISTS ix_promocode_activations_promo ON promocode_activations (promocode_id);
""
""
CREATE INDEX IF NOT EXISTS ix_promocode_activations_user ON promocode_activations (user_id);
""
]

```

```
async def main():  
    async with engine.begin() as conn:  
        for sql in MIGRATIONS:  
            try:  
                await conn.execute(text(sql.strip()))  
            except Exception as e:  
                # SQLite ■■■■■■■■■■■■ PostgreSQL - ■■■■■■■■■■  
                log_info(logger, f"Migration skipped: {e}")  
await engine.dispose()  
log_info(logger, "Migrations applied successfully")
```

```
=====
FILE: app\models.py
=====
```

```
class Base(DeclarativeBase):
    pass
```

```
id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
telegram_id: Mapped[int] = mapped_column(BigInteger, unique=True, nullable=False, index=True)
username: Mapped[str | None] = mapped_column(String(255), nullable=True, index=True)
display_name: Mapped[str | None] = mapped_column(String(32), nullable=True, index=True)
first_name: Mapped[str | None] = mapped_column(String(255), nullable=True)
last_name: Mapped[str | None] = mapped_column(String(255), nullable=True)
phone_number: Mapped[str | None] = mapped_column(String(50), nullable=True)
balance: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
status: Mapped[str] = mapped_column(String(50), default="active")
is_admin: Mapped[bool] = mapped_column(Boolean, default=False)
agreed_to_rules: Mapped[bool] = mapped_column(Boolean, default=False)
nickname_set: Mapped[bool] = mapped_column(Boolean, default=False)
referral_code: Mapped[str | None] = mapped_column(String(50), unique=True, nullable=True)
referred_by_user_id: Mapped[int | None] = mapped_column(
    ForeignKey("users.id"), nullable=True
)
referral_earnings: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
last_bonus_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
bonus_streak: Mapped[int] = mapped_column(Integer, default=0)
xp: Mapped[int] = mapped_column(Integer, default=0)
level: Mapped[int] = mapped_column(Integer, default=1)
vip_until: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
promo_created_this_month: Mapped[int] = mapped_column(Integer, default=0)
created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

# ██████████
videos: Mapped[List["Video"]] = relationship(back_populates="uploader")
views: Mapped[List["VideoView"]] = relationship(back_populates="user")
ratings: Mapped[List["VideoRating"]] = relationship(back_populates="user")
payments: Mapped[List["Payment"]] = relationship(back_populates="user")
comments: Mapped[List["Comment"]] = relationship(back_populates="user")
reactions: Mapped[List["ContentReaction"]] = relationship(back_populates="user")
game_history: Mapped[List["GameHistory"]] = relationship(back_populates="user")
quest_progress: Mapped[List["DailyQuestProgress"]] = relationship(back_populates="user")
game_sessions: Mapped[List["GameSession"]] = relationship(back_populates="user")
action_logs: Mapped[List["UserActionLog"]] = relationship(back_populates="user")
balance_logs: Mapped[List["BalanceLog"]] = relationship(back_populates="user")
user_offers: Mapped[List["Offer"]] = relationship(back_populates="creator")
rentals: Mapped[List["OfferRental"]] = relationship(
    back_populates="renter", foreign_keys="OfferRental.renter_user_id"
)
ad_state: Mapped["UserAdState"] = relationship(back_populates="user", uselist=False)
created_promocodes: Mapped[List["Promocode"]] = relationship(back_populates="creator")
activated_promocodes: Mapped[List["PromocodeActivation"]] = relationship(back_populates="user")
```

```
class UserActionLog(Base):
    tablename = "user action logs"
```



```

created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

user: Mapped["User"] = relationship(back_populates="action_logs")

class BalanceLog(Base):
    __tablename__ = "balance_logs"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    amount: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    balance_before: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    balance_after: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    source: Mapped[str] = mapped_column(String(100), nullable=False)
    source_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    admin_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    details: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, index=True)

    user: Mapped["User"] = relationship(back_populates="balance_logs")

class Video(Base):
    __tablename__ = "videos"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    uploader_user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    content_type: Mapped[str] = mapped_column(String(20), default="video")
    telegram_file_id: Mapped[str] = mapped_column(Text, nullable=False)
    telegram_file_unique_id: Mapped[str] = mapped_column(String(255), nullable=False, index=True)
    status: Mapped[str] = mapped_column(String(20), default="pending")
    duration_seconds: Mapped[int | None] = mapped_column(Integer, nullable=True)
    file_size: Mapped[int | None] = mapped_column(BigInteger, nullable=True)
    rejection_reason: Mapped[str | None] = mapped_column(String(255), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    uploader: Mapped["User"] = relationship(back_populates="videos")
    views: Mapped[List["VideoView"]] = relationship(back_populates="video")
    ratings: Mapped[List["VideoRating"]] = relationship(back_populates="video")
    comments: Mapped[List["Comment"]] = relationship(back_populates="video")
    reactions: Mapped[List["ContentReaction"]] = relationship(back_populates="video")

class VideoView(Base):
    __tablename__ = "video_views"
    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_view"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="views")
    video: Mapped["Video"] = relationship(back_populates="views")

class VideoRating(Base):
    __tablename__ = "video_ratings"
    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_rating"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    rating: Mapped[int] = mapped_column(Integer, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="ratings")
    video: Mapped["Video"] = relationship(back_populates="ratings")

class Comment(Base):
    __tablename__ = "comments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    text: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="comments")
    video: Mapped["Video"] = relationship(back_populates="comments")

class ContentReaction(Base):

```

```

__tablename__ = "content_reactions"
__table_args__ = (
    UniqueConstraint("user_id", "video_id", name="uq_user_video_reaction"),
)

id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
reaction_type: Mapped[str] = mapped_column(String(10), nullable=False)
created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

user: Mapped["User"] = relationship(back_populates="reactions")
video: Mapped["Video"] = relationship(back_populates="reactions")

class Payment(Base):
    __tablename__ = "payments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    payload: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    stars_amount: Mapped[int] = mapped_column(Integer, nullable=False)
    coins_amount: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="pending")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="payments")

class Offer(Base):
    __tablename__ = "offers"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    creator_user_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"), nullable=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    channel_url: Mapped[str] = mapped_column(Text, nullable=False)
    reward_preview: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("5"))
    reward_final: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("35"))
    penalty_unsubscribe: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("40"))
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    status: Mapped[str] = mapped_column(String(20), default="approved")
    is_rentable: Mapped[bool] = mapped_column(Boolean, default=False)
    rent_cost_per_day: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
    max_simultaneous_rentals: Mapped[int] = mapped_column(Integer, default=1)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    creator: Mapped["User"] = relationship(back_populates="user_offers")
    participations: Mapped[List["OfferParticipation"]] = relationship(back_populates="offer")
    rentals: Mapped[List["OfferRental"]] = relationship(back_populates="offer")

class OfferParticipation(Base):
    __tablename__ = "offer_participations"
    __table_args__ = (
        UniqueConstraint("user_id", "offer_id", name="uq_user_offer"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    offer_id: Mapped[int] = mapped_column(ForeignKey("offers.id"), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="started")
    reward_given: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    checked_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    offer: Mapped["Offer"] = relationship(back_populates="participations")

class OfferRental(Base):
    __tablename__ = "offer_rentals"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    offer_id: Mapped[int] = mapped_column(ForeignKey("offers.id"), nullable=False, index=True)
    renter_user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    renter_channel_title: Mapped[str] = mapped_column(String(255), nullable=False)
    renter_channel_url: Mapped[str] = mapped_column(Text, nullable=False)
    rent_days: Mapped[int] = mapped_column(Integer, nullable=False)
    cost_paid: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="pending")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    expires_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    renter: Mapped["User"] = relationship(
        back_populates="rentals", foreign_keys=[renter_user_id]
    )
    offer: Mapped["Offer"] = relationship(back_populates="rentals")

```

```

class GameHistory(Base):
    __tablename__ = "game_history"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    game_type: Mapped[str] = mapped_column(String(50), nullable=False)
    bet: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
    result: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
    details: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, index=True)

    user: Mapped["User"] = relationship(back_populates="game_history")

class DailyQuestProgress(Base):
    __tablename__ = "daily_quest_progress"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    quest_type: Mapped[str] = mapped_column(String(50), nullable=False)
    quest_date: Mapped[datetime] = mapped_column(Date, nullable=False)
    progress: Mapped[int] = mapped_column(Integer, default=0)
    target: Mapped[int] = mapped_column(Integer, nullable=False)
    reward: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=Decimal("0"))
    completed: Mapped[bool] = mapped_column(Boolean, default=False)
    reward_claimed: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="quest_progress")

class GameSession(Base):
    __tablename__ = "game_sessions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    window_start: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    games_played: Mapped[int] = mapped_column(Integer, default=0)
    paid_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    user: Mapped["User"] = relationship(back_populates="game_sessions")

class UserAdState(Base):
    __tablename__ = "user_ad_states"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(
        ForeignKey("users.id"), nullable=False, unique=True, index=True
    )
    last_offer_shown_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    last_low_balance_hint_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    forced_offer_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    forced_offer_shown_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="ad_state")

# =====
# ██████████ (██████ ██████████)
# =====
class Promocode(Base):
    """██████████, ██████████ ████████████████████ ███ Stars."""
    __tablename__ = "promocodes"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    creator_user_id: Mapped[int] = mapped_column(
        ForeignKey("users.id"), nullable=False, index=True
    )
    code: Mapped[str] = mapped_column(String(50), unique=True, nullable=False, index=True)
    coin_amount: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    max_uses: Mapped[int] = mapped_column(Integer, nullable=False)
    used_count: Mapped[int] = mapped_column(Integer, default=0)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    is_public: Mapped[bool] = mapped_column(Boolean, default=False)
    created_via_stars: Mapped[bool] = mapped_column(Boolean, default=True) # False █████ ████████████████████
    # ██████████ Stars ██████████
    stars_paid: Mapped[int] = mapped_column(Integer, default=0)
    expires_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    creator: Mapped["User"] = relationship(back_populates="created_promocodes")
    activations: Mapped[List["PromocodeActivation"]] = relationship(back_populates="promocode")

class PromocodeActivation(Base):

```



```

        if user and user.is_admin:
            return True
        return False

# =====
# ██████████ ████████ █ ██████████
# =====
async def log_balance_change(
    session: AsyncSession,
    user: "User",
    amount: Decimal,
    source: str,
    source_id: int = None,
    admin_id: int = None,
    details: str = None,
):
    log = BalanceLog(
        user_id=user.id,
        amount=amount,
        balance_before=user.balance,
        balance_after=user.balance + amount,
        source=source,
        source_id=source_id,
        admin_id=admin_id,
        details=details,
    )
    session.add(log)

async def log_user_action(
    session: AsyncSession,
    user_id: int,
    action: str,
    details: str = None
):
    log = UserActionLog(user_id=user_id, action=action, details=details)
    session.add(log)
    try:
        await session.commit()
    except Exception:
        await session.rollback()

# =====
# ██████████ ████████████████████
# =====
async def get_user(session: AsyncSession, telegram_id: int) -> "User | None":
    return (await session.execute(
        select(User).where(User.telegram_id == telegram_id)
    )).scalar_one_or_none()

async def get_user_by_id(session: AsyncSession, user_id: int) -> "User | None":
    return (await session.execute(
        select(User).where(User.id == user_id)
    )).scalar_one_or_none()

async def get_user_by_username(session: AsyncSession, username: str) -> "User | None":
    if username.startswith("@"):
        username = username[1:]
    return (await session.execute(
        select(User).where(User.username == username)
    )).scalar_one_or_none()

async def get_user_by_display_name(session: AsyncSession, display_name: str) -> "User | None":
    return (await session.execute(
        select(User).where(User.display_name == display_name)
    )).scalar_one_or_none()

async def get_or_create_user(
    session: AsyncSession,
    telegram_id: int,
    username: str = None,
    first_name: str = None,
    last_name: str = None,
    referral_code: str = None,
) -> tuple["User", bool]:
    user = await get_user(session, telegram_id)
    if user:
        user.username = username
        user.first_name = first_name
        user.last_name = last_name
        await session.commit()
        return user, False

```

```

referred_by = None
starting_bonus = to_decimal(STARTING_BALANCE)
if referral_code:
    inviter = (await session.execute(
        select(User).where(User.referral_code == referral_code)
    )).scalar_one_or_none()
    if inviter and inviter.telegram_id != telegram_id:
        referred_by = inviter.id

user = User(
    telegram_id=telegram_id,
    username=username,
    first_name=first_name,
    last_name=last_name,
    balance=starting_bonus,
    referral_code=uuid.uuid4().hex[:8],
    referred_by_user_id=referred_by,
)
session.add(user)
await session.flush()
await log_balance_change(session, user, starting_bonus, "registration",
    details=f"Starting balance. Referred by: {referred_by}")
await session.commit()
await log_user_action(session, user.id, "registration",
    f"tg_id={telegram_id}, referred_by={referred_by}")
return user, True

# =====
# ████████
# =====
async def set_display_name(session: AsyncSession, user: "User", name: str) -> tuple[bool, str]:
    import re
    name = name.strip()
    if len(name) < NICKNAME_MIN_LENGTH:
        return False, f"████████ ██████████. ██████████ {NICKNAME_MIN_LENGTH} ██████████."
    if len(name) > NICKNAME_MAX_LENGTH:
        return False, f"████████ ██████████. ██████████ {NICKNAME_MAX_LENGTH} ██████████."
    if not re.match(r'^[a-zA-Z█-█-█-0-9_\-]+$' , name):
        return False, "████████ ██████████ ██████████ ██████████ ██████████, ██████████, _ █ -"
    existing = await get_user_by_display_name(session, name)
    if existing and existing.id != user.id:
        return False, "████████ ███ ██████████."

    is_first = not user.nickname_set
    if not is_first:
        cost = to_decimal(NICKNAME_CHANGE_COST)
        if user.balance < cost:
            return False, f"████████████████████ ██████████. ██████████: {cost}, █ ██████: {user.balance}"
        await log_balance_change(session, user, -cost, "nickname_change",
            details=f"{user.display_name} -> {name}")
        user.balance -= cost

    old_name = user.display_name
    user.display_name = name
    user.nickname_set = True
    await session.commit()
    await log_user_action(session, user.id, "set_nickname", f"{old_name} -> {name}")
    if is_first:
        return True, f"████████ <b>{name}</b> ████████████████████████████████████████!"
    return True, f"████████ ██████████ ███ <b>{name}</b>! ██████████ {NICKNAME_CHANGE_COST} ██████████."

# =====
# ████████ / ████████
# =====
async def get_next_pending_video(session: AsyncSession) -> "Video | None":
    return (await session.execute(
        select(Video)
        .where(Video.status == "pending")
        .order_by(Video.created_at)
        .limit(1)
    )).scalar_one_or_none()

async def count_pending_videos(session: AsyncSession) -> int:
    return (await session.execute(
        select(func.count(Video.id)).where(Video.status == "pending")
    )).scalar_one()

async def count_approved_videos(session: AsyncSession) -> int:
    return (await session.execute(
        select(func.count(Video.id)).where(Video.status == "approved")
    )).scalar_one()

```

```

async def count_rejected_videos(session: AsyncSession) -> int:
    return (await session.execute(
        select(func.count(Video.id)).where(Video.status == "rejected")
    )).scalar_one()

async def approve_video(session: AsyncSession, video_id: int) -> "Video | None":
    v = (await session.execute(
        select(Video).where(Video.id == video_id)
    )).scalar_one_or_none()
    if not v or v.status != "pending":
        return None
    v.status = "approved"
    uploader = await get_user_by_id(session, v.uploader_user_id)
    if uploader:
        reward = PHOTO_UPLOAD_REWARD if v.content_type == "photo" else to_decimal(UPLOAD_REWARD)
        await log_balance_change(session, uploader, reward, "upload_approved", source_id=v.id)
        uploader.balance += reward
        await log_user_action(session, uploader.id, "video_approved",
            f"video_id={v.id}, reward={reward}")
    await session.commit()
    return v

async def reject_video(session: AsyncSession, video_id: int, reason: str) -> "Video | None":
    v = (await session.execute(
        select(Video).where(Video.id == video_id)
    )).scalar_one_or_none()
    if not v:
        return None
    v.status = "rejected"
    v.rejection_reason = reason
    await session.commit()
    await log_user_action(session, v.uploader_user_id, "video_rejected",
        f"video_id={v.id}, reason={reason}")
    return v

async def save_video(session: AsyncSession, user_id: int, file_id: str,
    file_unique_id: str, duration: int = None,
    file_size: int = None) -> tuple["Video", bool]:
    existing = (await session.execute(
        select(Video).where(Video.telegram_file_unique_id == file_unique_id)
    )).scalar_one_or_none()
    if existing:
        return existing, True
    video = Video(
        uploader_user_id=user_id,
        content_type="video",
        telegram_file_id=file_id,
        telegram_file_unique_id=file_unique_id,
        duration_seconds=duration,
        file_size=file_size,
        status="pending",
    )
    session.add(video)
    await session.commit()
    await log_user_action(session, user_id, "upload_video", f"file={file_unique_id}")
    return video, False

async def save_photo(session: AsyncSession, user_id: int, file_id: str,
    file_unique_id: str, file_size: int = None) -> tuple["Video", bool]:
    existing = (await session.execute(
        select(Video).where(Video.telegram_file_unique_id == file_unique_id)
    )).scalar_one_or_none()
    if existing:
        return existing, True
    photo = Video(
        uploader_user_id=user_id,
        content_type="photo",
        telegram_file_id=file_id,
        telegram_file_unique_id=file_unique_id,
        file_size=file_size,
        status="pending",
    )
    session.add(photo)
    await session.commit()
    await log_user_action(session, user_id, "upload_photo", f"file={file_unique_id}")
    return photo, False

async def get_random_video_for_user(session: AsyncSession, user_id: int) -> "Video | None":
    viewed = select(VideoView.video_id).where(VideoView.user_id == user_id)
    return (await session.execute(
        select(Video).where(
            Video.status == "approved",
            Video.content_type == "video",

```

```

        Video.uploader_user_id != user_id,
        ~Video.id.in_(viewed)
    ).order_by(func.random()).limit(1)
)).scalar_one_or_none()

async def get_random_photo_for_user(session: AsyncSession, user_id: int) -> "Video | None":
    return (await session.execute(
        select(Video).where(
            Video.status == "approved",
            Video.content_type == "photo",
            Video.uploader_user_id != user_id,
        ).order_by(func.random()).limit(1)
    )).scalar_one_or_none()

async def record_view_and_charge(session: AsyncSession, user_id: int, video_id: int) -> bool:
    user = await get_user_by_id(session, user_id)
    if not user:
        return False
    cost = to_decimal(WATCH_COST)
    if user.balance < cost:
        return False
    await log_balance_change(session, user, -cost, "watch", source_id=video_id)
    user.balance -= cost
    view = VideoView(user_id=user_id, video_id=video_id)
    session.add(view)
    await session.commit()
    return True

async def record_photo_view(session: AsyncSession, user_id: int, photo_id: int) -> bool:
    existing = (await session.execute(
        select(VideoView).where(
            VideoView.user_id == user_id, VideoView.video_id == photo_id
        )
    )).scalar_one_or_none()
    if not existing:
        view = VideoView(user_id=user_id, video_id=photo_id)
        session.add(view)
        await session.commit()
    return True

async def check_daily_photo_limit(session: AsyncSession, user_id: int) -> bool:
    today = datetime.utcnow().date()
    count = (await session.execute(
        select(func.count(VideoView.id)).where(
            VideoView.user_id == user_id,
            func.date(VideoView.created_at) == today,
            VideoView.video_id.in_(
                select(Video.id).where(Video.content_type == "photo")
            )
        )
    )).scalar_one()
    return count < DAILY_PHOTO_LIMIT

# =====
# ██████████
# =====
async def rate_video(session: AsyncSession, user_id: int, video_id: int, rating: int) -> bool:
    existing = (await session.execute(
        select(VideoRating).where(
            VideoRating.user_id == user_id, VideoRating.video_id == video_id
        )
    )).scalar_one_or_none()
    if existing:
        existing.rating = rating
    else:
        session.add(VideoRating(user_id=user_id, video_id=video_id, rating=rating))
    await session.commit()
    return True

# =====
# ██████████ ■ █████
# =====
async def update_user_balance(session: AsyncSession, user_id: int, amount: Decimal,
                             admin_id: int = None) -> bool:
    user = await get_user_by_id(session, user_id)
    if not user:
        return False
    await log_balance_change(session, user, amount, "admin_balance", admin_id=admin_id,
                             details=f"Manual by admin {admin_id}")
    user.balance += amount
    await log_user_action(session, user_id, "balance_update",
                          f"admin={admin_id}, amount={amount}")

```



```

await session.commit()
return True

async def set_user_ban_status(session: AsyncSession, user_id: int, is_banned: bool,
                             admin_id: int) -> bool:
    user = await get_user_by_id(session, user_id)
    if not user:
        return False
    user.status = "banned" if is_banned else "active"
    await log_user_action(session, user_id, "ban_status_change",
                          f"admin={admin_id}, banned={is_banned}")
    await session.commit()
    return True

# =====
# ██████████ ██████ (██████████████████)
# =====
async def claim_daily_bonus(session: AsyncSession, user_id: int) -> tuple[bool, str]:
    user = await get_user_by_id(session, user_id)
    if not user:
        return False, "██████████████████ ██ ████████."
    now = datetime.utcnow()
    if user.last_bonus_at and user.last_bonus_at.date() == now.date():
        return False, "██ █████ ██████████ █████ ████████."
    streak = 1
    if user.last_bonus_at and (now.date() - user.last_bonus_at.date()).days == 1:
        streak = min(user.bonus_streak + 1, MAX_BONUS_STREAK)
    reward = DAILY_BONUS_STREAK_BASE + DAILY_BONUS_STREAK_INCREASE * (streak - 1)
    await log_balance_change(session, user, to_decimal(reward), "daily_bonus")
    user.balance += to_decimal(reward)
    user.last_bonus_at = now
    user.bonus_streak = streak
    await session.commit()
    return True, f"██████████████████ ██████: +{reward:.0f} ██████! █████ ████████: {streak}"

# =====
# ██████████
# =====
async def count_referrals(session: AsyncSession, user_id: int) -> int:
    return (await session.execute(
        select(func.count(User.id)).where(User.referred_by_user_id == user_id)
    )).scalar_one()

async def process_referral_reward(session: AsyncSession, referrer_id: int):
    """██████████████████ ██████, █████ ██████████ ██████████ 5 ██████."""
    refs = (await session.execute(
        select(User).where(User.referred_by_user_id == referrer_id)
    )).scalars().all()
    for ref in refs:
        views = (await session.execute(
            select(func.count(VideoView.id)).where(VideoView.user_id == ref.id)
        )).scalar_one()
        if views >= 5:
            inviter = await get_user_by_id(session, referrer_id)
            if inviter:
                already = (await session.execute(
                    select(BalanceLog).where(
                        BalanceLog.user_id == referrer_id,
                        BalanceLog.source == "referral_reward",
                        BalanceLog.details == f"ref_user_id={ref.id}"
                    )
                )).scalar_one_or_none()
                if not already:
                    reward = to_decimal(REFERRAL_REWARD_INVITER)
                    await log_balance_change(session, inviter, reward, "referral_reward",
                                             details=f"ref_user_id={ref.id}")
                    inviter.balance += reward
                    inviter.referral_earnings += reward
                    await session.commit()

# =====
# ██████████
# =====
async def create_payment(session: AsyncSession, user_id: int, pack_key: str) -> Payment:
    pack = STARS_PACKAGES.get(pack_key)
    if not pack:
        raise ValueError("Unknown pack")
    coins = to_decimal(pack["coins"])
    payload = f"{pack_key}_{user_id}_{uuid.uuid4().hex[:6]}"
    payment = Payment(
        user_id=user_id,
        payload=payload,
        stars_amount=pack["stars"],

```

```

        coins_amount=coins,
        status="pending",
    )
    session.add(payment)
    await session.commit()
    return payment

async def create_custom_payment(session: AsyncSession, user_id: int, stars: int) -> Payment:
    coins = to_decimal(stars * STARS_TO_COINS_RATE)
    payload = f"custom_{user_id}_{uuid.uuid4().hex[:6]}"
    payment = Payment(
        user_id=user_id,
        payload=payload,
        stars_amount=stars,
        coins_amount=coins,
        status="pending",
    )
    session.add(payment)
    await session.commit()
    return payment

async def apply_successful_payment(session: AsyncSession, payload: str) -> Payment | None:
    payment = (await session.execute(
        select(Payment).where(Payment.payload == payload)
    )).scalar_one_or_none()
    if not payment or payment.status != "pending":
        return None
    payment.status = "paid"
    user = await get_user_by_id(session, payment.user_id)
    if user:
        bonus_multiplier = 1.0
        # *****
        if DYNAMIC_STAR_DISCOUNT_ENABLED:
            try:
                start_h, end_h = map(int, DYNAMIC_STAR_DISCOUNT_HOURS.split("-"))
                now_h = datetime.utcnow().hour
                if start_h <= now_h < end_h:
                    bonus_multiplier = DYNAMIC_STAR_DISCOUNT_MULTIPLIER
            except Exception:
                pass
        # *****
        today = datetime.utcnow().date()
        first_today = not (await session.execute(
            select(Payment).where(
                Payment.user_id == user.id,
                Payment.status == "paid",
                func.date(Payment.created_at) == today,
                Payment.id != payment.id,
            )
        )).scalar_one_or_none()
        total_coins = payment.coins_amount * to_decimal(bonus_multiplier)
        if first_today:
            total_coins += to_decimal(FIRST_PURCHASE_DAILY_BONUS)
        await log_balance_change(session, user, total_coins, "purchase",
                                details=f"payload={payload}, bonus_mult={bonus_multiplier}, first_today={first_t
oday}")
        user.balance += total_coins
        await session.commit()
    return payment

# =====
# *****
# =====

async def get_active_offers(session: AsyncSession) -> list["Offer"]:
    return (await session.execute(
        select(Offer).where(Offer.is_active == True, Offer.status == "approved")
    )).scalars().all()

async def get_rentable_offers(session: AsyncSession) -> list["Offer"]:
    return (await session.execute(
        select(Offer).where(
            Offer.is_active == True,
            Offer.status == "approved",
            Offer.is_rentable == True,
        )
    )).scalars().all()

async def get_offer_by_id(session: AsyncSession, offer_id: int) -> "Offer | None":
    return (await session.execute(
        select(Offer).where(Offer.id == offer_id)
    )).scalar_one_or_none()

```

```

async def start_offer_participation(session: AsyncSession, user_id: int,
                                   offer_id: int) -> tuple["OfferParticipation | None", bool]:
    existing = (await session.execute(
        select(OfferParticipation).where(
            OfferParticipation.user_id == user_id,
            OfferParticipation.offer_id == offer_id,
        )
    )).scalar_one_or_none()
    if existing:
        return existing, False
    part = OfferParticipation(user_id=user_id, offer_id=offer_id, status="started")
    session.add(part)
    await session.commit()
    return part, True


async def verify_offer_subscription(session: AsyncSession, user_id: int,
                                   offer_id: int) -> bool:
    part = (await session.execute(
        select(OfferParticipation).where(
            OfferParticipation.user_id == user_id,
            OfferParticipation.offer_id == offer_id,
        )
    )).scalar_one_or_none()
    if not part:
        return False
    if part.status == "completed":
        return True
    part.status = "completed"
    part.checked_at = datetime.utcnow()
    offer = await get_offer_by_id(session, offer_id)
    user = await get_user_by_id(session, user_id)
    if offer and user:
        additional = offer.reward_final - part.reward_given
        if additional > 0:
            await log_balance_change(session, user, additional, "offer_complete", source_id=offer_id)
            user.balance += additional
            part.reward_given = offer.reward_final
    await session.commit()
    return True


async def admin_create_offer(session: AsyncSession, title: str, description: str,
                             channel_url: str, reward_preview: Decimal,
                             reward_final: Decimal, is_rentable: bool = False,
                             rent_cost_per_day: Decimal = Decimal("0"),
                             max_simultaneous_rentals: int = 1) -> "Offer":
    offer = Offer(
        creator_user_id=None,
        title=title,
        description=description,
        channel_url=channel_url,
        reward_preview=reward_preview,
        reward_final=reward_final,
        is_active=True,
        status="approved",
        is_rentable=is_rentable,
        rent_cost_per_day=rent_cost_per_day,
        max_simultaneous_rentals=max_simultaneous_rentals,
    )
    session.add(offer)
    await session.commit()
    return offer


async def count_active_rentals(session: AsyncSession) -> int:
    try:
        return (await session.execute(
            select(func.count(OfferRental.id)).where(OfferRental.status == "active")
        )).scalar_one()
    except Exception:
        return 0


async def expire_old_rentals(session: AsyncSession) -> int:
    try:
        now = datetime.utcnow()
        expired = (await session.execute(
            select(OfferRental).where(
                OfferRental.status == "active",
                OfferRental.expires_at <= now
            )
        )).scalars().all()
        for r in expired:
            r.status = "expired"
        await session.commit()
        return len(expired)
    except Exception:

```

```
return 0
```

[illegible]

```

async def pay_for_game_session(session: AsyncSession, user_id: int) -> bool:
    user = await get_user_by_id(session, user_id)
    if not user:
        return False
    cost = to_decimal(GAME_SESSION_COST)
    if user.balance < cost:
        return False
    await log_balance_change(session, user, -cost, "game_session_paid")
    user.balance -= cost
    gs = await get_or_create_game_session(session, user_id)
    gs.games_played = 0
    gs.window_start = datetime.utcnow()
    gs.paid_at = datetime.utcnow()
    await session.commit()
    return True

async def increment_game_played(session: AsyncSession, user_id: int):
    gs = await get_or_create_game_session(session, user_id)
    gs.games_played += 1
    await session.commit()

# =====
# ■■■■■■■■■■
# =====
async def get_admin_extended_stats(session: AsyncSession) -> dict:
    users = (await session.execute(select(func.count(User.id)))).scalar_one()
    vip = (await session.execute(
        select(func.count(User.id)).where(User.vip_until > datetime.utcnow())
    )).scalar_one()
    with_nickname = (await session.execute(
        select(func.count(User.id)).where(User.nickname_set == True)
    )).scalar_one()
    comments = (await session.execute(select(func.count(Comment.id)))).scalar_one()
    reactions = (await session.execute(select(func.count(ContentReaction.id)))).scalar_one()
    games = (await session.execute(select(func.count(GameHistory.id)))).scalar_one()
    offers = (await session.execute(select(func.count(Offer.id)))).scalar_one()
    total_balance = (await session.execute(
        select(func.sum(User.balance))
    )).scalar_one() or Decimal("0")
    total_admin_given = (await session.execute(
        select(func.sum(BalanceLog.amount)).where(
            BalanceLog.source == "admin_balance", BalanceLog.amount > 0
        )
    )).scalar_one() or Decimal("0")
    total_game_profit = (await session.execute(
        select(func.sum(GameHistory.result))
    )).scalar_one() or Decimal("0")
    try:
        active_rentals = (await session.execute(
            select(func.count(OfferRental.id)).where(OfferRental.status == "active")
        )).scalar_one()
        total_rent_income = (await session.execute(
            select(func.sum(BalanceLog.amount)).where(BalanceLog.source == "offer_rental")
        )).scalar_one() or Decimal("0")
    except Exception:
        active_rentals = 0
        total_rent_income = Decimal("0")
    return {
        "users": users, "vip": vip, "with_nickname": with_nickname,
        "comments": comments, "reactions": reactions, "games": games,
        "offers": offers, "active_rentals": active_rentals,
        "total_balance_in_system": total_balance,
        "total_admin_given": total_admin_given,
        "total_game_profit": total_game_profit,
        "total_rent_income": total_rent_income,
    }

async def get_user_dossier(session: AsyncSession, user_id: int) -> dict | None:
    user = await get_user_by_id(session, user_id)
    if not user:
        return None
    games_count = (await session.execute(
        select(func.count(GameHistory.id)).where(GameHistory.user_id == user_id)
    )).scalar_one()
    game_profit = (await session.execute(
        select(func.sum(GameHistory.result)).where(GameHistory.user_id == user_id)
    )).scalar_one() or Decimal("0")
    suspicious_games = (await session.execute(
        select(GameHistory).where(
            GameHistory.user_id == user_id, GameHistory.result > to_decimal(50)
        ).order_by(desc(GameHistory.created_at))
    )).scalars().all()
    videos_uploaded = (await session.execute(

```

```

        select(func.count(Video.id)).where(Video.uploader_user_id == user_id)
    )),scalar_one()
    videos_watched = (await session.execute(
        select(func.count(VideoView.id)).where(VideoView.user_id == user_id)
    )),scalar_one()
    comments_count = (await session.execute(
        select(func.count(Comment.id)).where(Comment.user_id == user_id)
    )),scalar_one()
    reactions_count = (await session.execute(
        select(func.count(ContentReaction.id)).where(ContentReaction.user_id == user_id)
    )),scalar_one()
    avg_rating = (await session.execute(
        select(func.avg(VideoRating.rating)).where(
            VideoRating.video_id.in_(
                select(Video.id).where(Video.uploader_user_id == user_id)
            )
        )
    )),scalar_one()
    avg_rating = float(avg_rating) if avg_rating else 0.0
    week_ago = datetime.utcnow() - timedelta(days=7)
    weekly_earned = (await session.execute(
        select(func.sum(BalanceLog.amount)).where(
            BalanceLog.user_id == user_id, BalanceLog.amount > 0,
            BalanceLog.created_at >= week_ago
        )
    )),scalar_one() or Decimal("0")
    weekly_spent = (await session.execute(
        select(func.sum(BalanceLog.amount)).where(
            BalanceLog.user_id == user_id, BalanceLog.amount < 0,
            BalanceLog.created_at >= week_ago
        )
    )),scalar_one() or Decimal("0")
    balance_logs = (await session.execute(
        select(BalanceLog).where(BalanceLog.user_id == user_id)
        .order_by(desc(BalanceLog.created_at)).limit(50)
    )),scalars().all()
    action_logs = (await session.execute(
        select(UserActionLog).where(UserActionLog.user_id == user_id)
        .order_by(desc(UserActionLog.created_at)).limit(20)
    )),scalars().all()
    total_earned = (await session.execute(
        select(func.sum(BalanceLog.amount)).where(
            BalanceLog.user_id == user_id, BalanceLog.amount > 0
        )
    )),scalar_one() or Decimal("0")
    total_spent = (await session.execute(
        select(func.sum(BalanceLog.amount)).where(
            BalanceLog.user_id == user_id, BalanceLog.amount < 0
        )
    )),scalar_one() or Decimal("0")
    admin_given = (await session.execute(
        select(func.sum(BalanceLog.amount)).where(
            BalanceLog.user_id == user_id,
            BalanceLog.source == "admin_balance", BalanceLog.amount > 0
        )
    )),scalar_one() or Decimal("0")
    return {
        "user": user, "games_count": games_count,
        "game_profit": game_profit, "suspicious_games": suspicious_games,
        "videos_uploaded": videos_uploaded, "videos_watched": videos_watched,
        "comments_count": comments_count, "reactions_count": reactions_count,
        "avg_rating": round(avg_rating, 2),
        "weekly_earned": weekly_earned, "weekly_spent": weekly_spent,
        "balance_logs": balance_logs, "action_logs": action_logs,
        "total_earned": total_earned, "total_spent": total_spent,
        "admin_given": admin_given,
    }

# =====
# ■■■■■■■■■■
# =====
def generate_promocode_str(length: int = 10) -> str:
    chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    return ''.join(random.choices(chars, k=length))

def calculate_promocode_star_cost(coin_amount: Decimal, max_uses: int) -> int:
    """■■■■■■■■ ■■■■■■■■■ ■■■■■■■■■ ■ Stars."""
    base = float(coin_amount) * max_uses * PROMOCODE_CREATION_STAR_RATE
    if max_uses >= PROMOCODE_BULK_DISCOUNT_THRESHOLD:
        base *= PROMOCODE_BULK_DISCOUNT_RATE
    return max(1, int(base))

async def create_promocode(
    session: AsyncSession,
    creator_tg_id: int,

```

```

coin_amount: Decimal,
max_uses: int,
hours: int = 24,
is_public: bool = False,
admin_free: bool = False,
) -> tuple["Promocode | None", int, str | None]:
    """
    """
    admin_free=True - (***** Stars).
    (*****, ****_Stars, *****).
    """
    user = await get_user(session, creator_tg_id)
    if not user:
        return None, 0, "*****."

    if coin_amount <= 0 or coin_amount > PROMOCODE_MAX_AMOUNT:
        return None, 0, f"***** 1 {PROMOCODE_MAX_AMOUNT} *****."
    if max_uses < 1 or max_uses > PROMOCODE_MAX_USES:
        return None, 0, f"***** 1 {PROMOCODE_MAX_USES} *****."
    if hours < 1 or hours > PROMOCODE_MAX_HOURS:
        return None, 0, f"***** 1 {PROMOCODE_MAX_HOURS} *****."

    star_cost = calculate_promocode_star_cost(coin_amount, max_uses)
    is_admin = is_admin_or_super(creator_tg_id, user)

    if not admin_free:
        # ***** VIP (***** )
        if user.vip_until and user.vip_until > datetime.utcnow():
            this_month = datetime.utcnow().month
            promo_month = user.promo_created_this_month
            # *****
            if promo_month != this_month:
                user.promo_created_this_month = 0
            if user.promo_created_this_month < VIP_FREE_PROMO_PER_MONTH:
                star_cost = 0
                user.promo_created_this_month += 1

        # ***** -> ***** (***** )
        if star_cost > 0:
            # ***** , *****
            pass

    code = generate_promocode_str()
    expires_at = datetime.utcnow() + timedelta(hours=hours)

    promo = Promocode(
        creator_user_id=user.id,
        code=code,
        coin_amount=to_decimal(coin_amount),
        max_uses=max_uses,
        used_count=0,
        is_active=True,
        is_public=is_public,
        created_via_stars=(star_cost > 0 and not admin_free),
        stars_paid=0 if (admin_free or (star_cost == 0 and user.vip_until)) else star_cost,
        expires_at=expires_at,
    )
    session.add(promo)
    await session.commit()
    await log_user_action(session, user.id, "create_promocode",
        f"code={code}, amount={coin_amount}, uses={max_uses}, admin_free={admin_free}")
    return promo, star_cost, None

```

```

async def activate_promocode(session: AsyncSession, user_id: int, code: str) -> str:

```

```

    """***** ."""
    user = await get_user_by_id(session, user_id)
    if not user:
        return "*****."
    promo = (await session.execute(
        select(Promocode).where(Promocode.code == code.upper().strip())
    )).scalar_one_or_none()
    if not promo:
        return "*****."
    if not promo.is_active:
        return "*****."
    if promo.expires_at and promo.expires_at < datetime.utcnow():
        return "*****."
    if promo.used_count >= promo.max_uses:
        return "*****."
    # *****
    already = (await session.execute(
        select(PromocodeActivation).where(
            PromocodeActivation.promocode_id == promo.id,
            PromocodeActivation.user_id == user_id,
        )
    )).scalar_one_or_none()
    if already:

```

[illegible]


```

import re
import uuid
import asyncio
from datetime import datetime, timedelta
from decimal import Decimal

from aiogram import Router, F
from aiogram.filters import CommandStart, CommandObject
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import (
    Message, CallbackQuery,
    LabeledPrice, PreCheckoutQuery,
    InlineKeyboardMarkup, InlineKeyboardButton,
)
from sqlalchemy import select, func, desc

from app.config import (
    ADMINS, WATCH_COST, STARS_PACKAGES, STARS_TO_COINS_RATE,
    XP_PER_WATCH, XP_PER_UPLOAD, XP_PER_RATING,
    XP_PER_COMMENT, XP_PER_REACTION, XP_PER_GAME,
    PIN_OFFER_COST, VIP_PRICE_STARS, VIP_DURATION_DAYS, VIP_BONUS_MULTIPLIER,
    LEVEL_XP_BASE, LEVEL_XP_MULTIPLIER,
    DAILY_QUESTS, PREMIUM_DAILY_QUESTS,
    COMMENTS_PER_10_MIN,
    NICKNAME_CHANGE_COST, NICKNAME_MIN_LENGTH, NICKNAME_MAX_LENGTH,
    OFFER_DEFAULT_RENT_COST_PER_DAY, OFFER_MIN_RENT_DAYS, OFFER_MAX_RENT_DAYS,
    REFERRAL_REWARD_INVITER, REFERRAL_REWARD_NEW_USER, SMART_AD_FORCED_WATCH_SECONDS,
    DAILY_PHOTO_LIMIT,
    FREE_GAMES_PER_SESSION, GAME_SESSION_HOURS, GAME_SESSION_COST,
    PROMOCODE_CREATION_STAR_RATE,
    PROMOCODE_BULK_DISCOUNT_THRESHOLD,
    PROMOCODE_BULK_DISCOUNT_RATE,
    PROMOCODE_CREATOR_BONUS_PERCENT,
    PROMOCODE_MAX_AMOUNT, PROMOCODE_MAX_USES, PROMOCODE_MAX_HOURS,
    VIP_FREE_PROMO_PER_MONTH,
    DYNAMIC_STAR_DISCOUNT_ENABLED,
    DYNAMIC_STAR_DISCOUNT_HOURS,
    DYNAMIC_STAR_DISCOUNT_MULTIPLIER,
    FIRST_PURCHASE_DAILY_BONUS,
)
from app.db import async_session
from app.models import (
    User, Video, VideoView, Comment, ContentReaction,
    DailyQuestProgress, GameHistory, OfferRental, Offer, Promocode,
)
from app.services import (
    get_or_create_user, get_user, get_user_by_id,
    save_video, save_photo,
    get_random_video_for_user, get_random_photo_for_user,
    record_view_and_charge, record_photo_view,
    rate_video, claim_daily_bonus, count_referrals,
    create_payment, create_custom_payment, apply_successful_payment,
    get_active_offers, get_rentable_offers, get_offer_by_id,
    start_offer_participation, verify_offer_subscription,
    create_offer_rental, get_user_rentals,
    log_user_action, to_decimal,
    set_display_name, get_display_name, log_balance_change,
    can_play_free_game, pay_for_game_session, increment_game_played,
    check_daily_photo_limit,
    create_promocode, activate_promocode,
    calculate_promocode_star_cost,
    is_admin_or_super,
    should_show_low_balance_hint, mark_low_balance_hint_shown,
    can_show_offer_to_user, mark_offer_shown,
    get_random_active_offer, should_inject_ad_in_video,
    process_referral_reward,
)
from app.keyboards import (
    rules_keyboard, main_menu,
    video_rating_keyboard, photo_actions_keyboard,
    watch_choice_keyboard, buy_coins_keyboard, vip_buy_keyboard,
    offers_list_keyboard, offer_view_keyboard,
    offer_rent_keyboard, rent_days_keyboard,
    games_menu_keyboard, tops_menu_keyboard,
    quests_keyboard, reaction_menu_keyboard,
    BTN_WATCH, BTN_UPLOAD, BTN_PROFILE, BTN_BUY,
    BTN_OFFERS, BTN_REFERRALS, BTN_BONUS, BTN_ADMIN,
    BTN_GAMES, BTN_TOPS, BTN_QUESTS, BTN_VIP, BTN_LEVEL,
    BTN_PROMO,
)
from app.logger import get_logger

logger = get_logger(__name__)
router = Router()

```

```

# =====

```

```

# STATES
# =====
class NicknameState(StatesGroup):
    waiting_nickname = State()

class CommentState(StatesGroup):
    waiting_text = State()

class CustomBuyState(StatesGroup):
    waiting_stars = State()

class RentOfferState(StatesGroup):
    waiting_channel_title = State()
    waiting_channel_url = State()
    waiting_days = State()

class PromoCreateState(StatesGroup):
    waiting_amount = State()
    waiting_uses = State()
    waiting_hours = State()

# =====
# HELPERS
# =====
def is_any_admin(telegram_id: int, user_obj=None) -> bool:
    if telegram_id in ADMINS:
        return True
    if user_obj and user_obj.is_admin:
        return True
    return False

def calc_level_xp_required(level: int) -> int:
    return int(LEVEL_XP_BASE * (LEVEL_XP_MULTIPLIER ** (level - 1)))

def calc_level_from_xp(xp: int) -> int:
    level = 1
    remaining = xp
    while True:
        required = calc_level_xp_required(level)
        if remaining < required:
            break
        remaining -= required
        level += 1
    return level

def is_vip(user) -> bool:
    return bool(user.vip_until and user.vip_until > datetime.utcnow())

async def require_nickname(message: Message, user) -> bool:
    """[REDACTED] False = [REDACTED], [REDACTED]."""
    if user.nickname_set and user.display_name:
        return True
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text="[REDACTED]",
            callback_data="set_nickname_start"
        )]
    ])
    await message.answer(
        f"[REDACTED] <b>[REDACTED] [REDACTED]!</b>\n\n"
        f"• [REDACTED] {NICKNAME_MIN_LENGTH} [REDACTED] {NICKNAME_MAX_LENGTH} [REDACTED]\n"
        f"• [REDACTED] [REDACTED] ([REDACTED]/[REDACTED]), [REDACTED], _ [REDACTED] -\n"
        f"• [REDACTED]\n\n"
        f"[REDACTED] [REDACTED] - [REDACTED]!\n"
        f"[REDACTED] [REDACTED] {NICKNAME_CHANGE_COST} [REDACTED].",
        parse_mode="HTML",
        reply_markup=kb
    )
    return False

async def _level_up_check(session, user, message_or_callback):
    """[REDACTED] [REDACTED] [REDACTED] [REDACTED]."""
    new_level = calc_level_from_xp(user.xp)
    if new_level > user.level:
        user.level = new_level
        await session.commit()
        if hasattr(message_or_callback, "answer"):
            await message_or_callback.answer(

```

```
f"""! {} <b>{new_level}</b>! ", parse_mode="HTML")
)
else:
    await message_or_callback.message.answer(
        f"! {} <b>{new_level}</b>!", parse_mode="HTML"
    )

# =====
# NICKNAME FLOW
# =====
@router.callback_query(F.data == "set_nickname_start")
async def set_nickname_start(callback: CallbackQuery, state: FSMContext):
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        is_first = not user.nickname_set
        cost_text = "" if is_first else f"{NICKNAME_CHANGE_COST} "

    await state.set_state(NicknameState.waiting_nickname)
    await callback.message.answer(
        f"— ({{cost_text}}):\n\n"
        f". {{NICKNAME_MIN_LENGTH}} {{NICKNAME_MAX_LENGTH}} \n"
        f". (/), , _ -\n"
        f". , , "
    )
    await callback.answer()

@router.message(NicknameState.waiting_nickname)
async def process_nickname(message: Message, state: FSMContext):
    name = (message.text or "").strip()
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            await state.clear()
            return
        ok, msg = await set_display_name(session, user, name)

    await message.answer(msg, parse_mode="HTML")
    if ok:
        await state.clear()
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            admin_flag = is_any_admin(message.from_user.id, user)
        await message.answer(
            "! ",
            reply_markup=main_menu(is_admin=admin_flag)
        )

# =====
# START / RULES
# =====
@router.message(CommandStart())
async def cmd_start(message: Message, command: CommandObject, state: FSMContext):
    if not message.from_user:
        return
    await state.clear()
    args = (command.args or "").strip()

    # (query promo_)
    if args.startswith("promo_"):
        promo_code = args.replace("promo_", "")
        async with async_session() as session:
            user, is_new = await get_or_create_user(
                session, message.from_user.id,
                message.from_user.username,
                message.from_user.first_name,
                message.from_user.last_name,
            )
        if user.status == "banned":
            await message.answer("! ")
            return
        result = await activate_promocode(session, user.id, promo_code)
        await message.answer(result)
        if not user.agreed_to_rules:
            await message.answer(
                "<b>\n\n"
                "1. . ( "
            ).\n"
                "2. , .\n"
                "3. , "
            ).\n\n"
```

[illegible]

```

    "■ ■■■■■■■■ ■■■■■■■■!\n\n"
    "■ ■■■■■■■■ ■■■■■■■■■■ ■■■. ■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■!\n"
    f"• ■■■ {NICKNAME_MIN_LENGTH} ■■■ {NICKNAME_MAX_LENGTH} ■■■■■■■■\n"
    f"• ■■■■■■■■ ■■■■■■■■ (■■■■/■■■■), ■■■■■, _ ■ -",
    parse_mode="HTML",
    reply_markup=kb
)
await callback.answer()

# =====
# ADMIN REDIRECT
# =====
@router.message(F.text == BTN_ADMIN)
async def cmd_admin_redirect(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if is_admin(message.from_user.id, user):
            from app.admin_handlers import cmd_admin
            await cmd_admin(message)

# =====
# PROFILE
# =====
@router.message(F.text == BTN_PROFILE)
async def show_profile(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return

        refs = await count_referrals(session, user.id)
        vip_str = ""
        if is_vip(user):
            vip_str = f"\n■ VIP ■■■: {user.vip_until.strftime('%d.%m.%Y')}}"

        level = user.level
        xp_spent = sum(calc_level_xp_required(l) for l in range(1, level))
        xp_current = user.xp - xp_spent
        xp_needed = calc_level_xp_required(level)
        progress = max(0, min(10, int((xp_current / max(xp_needed, 1)) * 10)))
        bar = "■" * progress + "■" * (10 - progress)

        kb = InlineKeyboardMarkup(inline_keyboard=[
            [InlineKeyboardButton(
                text="=■ ■■■■■■■■ ■■■",
                callback_data="set_nickname_start"
            )]
        ])
        text = (
            f"■ <b>■■■■■■■■</b>\n\n"
            f"■ ■■■: <b>{get_display_name(user)}</b>\n"
            f"■ TG ID: <code>{user.telegram_id}</code>\n"
            f"■ ■■■■■■■■: <b>{user.balance}</b> ■■■■■\n"
            f"■ ■■■■■■■■: <b>{user.level}</b>\n"
            f"■ XP: {xp_current}/{xp_needed} [{bar}]\n"
            f"■ ■■■■■■■■: {refs}\n"
            f"■ ■■■. ■■■■■■■■■■: {user.referral_earnings} ■■■■■\n"
            f"■ ■■■■■: {user.status}"
            f"{vip_str}\n\n"
            f"■ ■■■■■ ■■■: {NICKNAME_CHANGE_COST} ■■■■■"
        )
        await message.answer(text, parse_mode="HTML", reply_markup=kb)
        await log_user_action(session, user.id, "view_profile")

# =====
# LEVEL
# =====
@router.message(F.text == BTN_LEVEL)
async def show_level(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return

        level = user.level
        xp_spent = sum(calc_level_xp_required(l) for l in range(1, level))
        xp_current = user.xp - xp_spent
        xp_needed = calc_level_xp_required(level)
        progress = max(0, min(10, int((xp_current / max(xp_needed, 1)) * 10)))
        bar = "■" * progress + "■" * (10 - progress)

```

```

text = (
    f"■ <b>■■■■■■■■: {level}</b>\n\n"
    f"XP: {xp_current}/{xp_needed}\n"
    f"[{bar}]\n\n"
    f"■ ■■■ XP:\n"
    f"• ■■■■■: +{XP_PER_WATCH} XP\n"
    f"• ■■■■■: +{XP_PER_UPLOAD} XP\n"
    f"• ■■■■■: +{XP_PER_RATING} XP\n"
    f"• ■■■■■: +{XP_PER_COMMENT} XP\n"
    f"• ■■■■■: +{XP_PER_REACTION} XP\n"
    f"• ■■■: +{XP_PER_GAME} XP"
)
await message.answer(text, parse_mode="HTML")

# =====
# VIP
# =====
@router.message(F.text == BTN_VIP)
async def show_vip(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return

    if is_vip(user):
        await message.answer(
            f"■ <b>■ VIP!</b>\n\n"
            f"■■: <b>{user.vip_until.strftime('%d.%m.%Y %H:%M')}</b>\n\n"
            f"■■■■■■■■■■: \n"
            f"• ■■■■■■■■■■■■■■■■■ x{VIP_BONUS_MULTIPLIER}\n"
            f"• ■■■■■ 50% ■■■■■■■■■■■■■■■■■\n"
            f"• VIP ■■■■■■■■■■■■■■■■■",
            parse_mode="HTML"
        )
    else:
        await message.answer(
            f"■ <b>VIP ■■■■■■■■■■■■■■■■■</b>\n\n"
            f"■■■■■■■■■■: <b>{VIP_PRICE_STARS} Stars</b>\n"
            f"■■■■■■■■■■■■■■■■: {VIP_DURATION_DAYS} ■■■■■\n\n"
            f"■■■■■■■■■■■■■■■■: \n"
            f"• ■■■■■■■■■■■■■■■■■ x{VIP_BONUS_MULTIPLIER}\n"
            f"• ■■■■■ 50% ■■■■■■■■■■■■■■■■■\n"
            f"• VIP ■■■■■■■■■■■■■■■■■",
            parse_mode="HTML",
            reply_markup=vip_buy_keyboard()
        )

@router.callback_query(F.data == "buy_vip")
async def buy_vip(callback: CallbackQuery):
    await callback.message.answer_invoice(
        title="VIP ■■■■■■■■■■■■■■■■■",
        description=f"VIP ■■■ {VIP_DURATION_DAYS} ■■■■■",
        payload=f"vip_{callback.from_user.id}_{uuid.uuid4().hex[:6]}",
        currency="XTR",
        prices=[LabeledPrice(label="VIP", amount=VIP_PRICE_STARS)]
    )
    await callback.answer()

# =====
# WATCH
# =====
@router.message(F.text == BTN_WATCH)
async def btn_watch(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if user.status == "banned":
            await message.answer("■ ■■ ■■■■■■■■■■■■■■■■■")
            return
        if not await require_nickname(message, user):
            return
    await message.answer("■■■ ■■■■■■■■■■■■■■■?", reply_markup=watch_choice_keyboard())

@router.callback_query(F.data == "watch_video_content")
async def watch_video_content(callback: CallbackQuery):
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return

```

[illegible]

```

        caption=(
            f"📺 📺📺📺📺📺 #{video.id}\n"
            f"📺 📺📺📺📺📺: {cost} 📺📺📺📺"
        ),
        reply_markup=video_rating_keyboard(video.id)
    )
    await callback.answer()

@router.callback_query(F.data == "watch_next")
async def watch_next(callback: CallbackQuery):
    await watch_video_content(callback)

@router.callback_query(F.data == "watch_photo_content")
async def watch_photo_content(callback: CallbackQuery):
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return

        # 📺📺📺📺📺📺 📺📺📺📺📺📺 📺📺📺📺 📺📺 📺📺 📺📺📺📺📺📺📺📺📺📺
        if not is_vip(user):
            can_view = await check_daily_photo_limit(session, user.id)
            if not can_view:
                await callback.message.answer(
                    f"📺 📺 📺📺📺📺📺📺📺 📺📺📺📺📺📺 📺📺 📺📺📺📺📺 ({DAILY_PHOTO_LIMIT} 📺📺).\n"
                    f"📺📺 VIP 📺📺📺📺📺📺📺📺📺 📺📺📺📺📺📺📺📺📺📺📺.",
                    parse_mode="HTML"
                )
                await callback.answer()
                return

            photo = await get_random_photo_for_user(session, user.id)
            if not photo:
                await callback.message.answer("📺 📺📺 📺📺📺📺📺📺📺📺.")
                await callback.answer()
                return
            await record_photo_view(session, user.id, photo.id)

        await callback.message.answer_photo(
            photo.telegram_file_id,
            caption=f"📺 📺📺📺📺📺 #{photo.id}",
            reply_markup=photo_actions_keyboard(photo.id)
        )
    await callback.answer()

@router.callback_query(F.data == "watch_next_photo")
async def watch_next_photo(callback: CallbackQuery):
    await watch_photo_content(callback)

# =====
# RATING
# =====
@router.callback_query(F.data.startswith("rate:"))
async def cb_rate(callback: CallbackQuery):
    parts = callback.data.split(":")
    if len(parts) != 3:
        await callback.answer()
        return
    video_id, rating = int(parts[1]), int(parts[2])

    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        await rate_video(session, user.id, video_id, rating)
        user.xp += XP_PER_RATING
        await _level_up_check(session, user, callback)
        await _update_quest_progress(session, user.id, "rate", 1)

    await callback.answer(f"📺 📺📺📺📺📺 {rating} 📺📺📺📺📺!")

# =====
# COMMENTS
# =====
@router.callback_query(F.data.startswith("comments:"))
async def cb_comments(callback: CallbackQuery):
    video_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        comments = (await session.execute(
            select(Comment)
            .where(Comment.video_id == video_id)

```



```

        .order_by(desc(Comment.created_at))
        .limit(10)
    )).scalars().all()

    text = f"■ <b>■■■■■■■■■■ ■ ■■■■■ ■{video_id}</b>\n\n"
    if not comments:
        text += "■■■■■■■■■■ ■■■■ ■■■. ■■■■■ ■■■■■!"
    else:
        for c in comments:
            u = await get_user_by_id(session, c.user_id)
            name = get_display_name(u) if u else "???"
            text += f"■ <b>{name}</b>: {c.text}\n"

kb = InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(
        text="■ ■■■■■■■■",
        callback_data=f"add_comment:{video_id}"
    )],
    [InlineKeyboardButton(
        text="■ ■■■■■■■■",
        callback_data=f"reactions:{video_id}"
    )],
])
await callback.message.answer(text, parse_mode="HTML", reply_markup=kb)
await callback.answer()

@router.callback_query(F.data.startswith("add_comment:"))
async def add_comment_start(callback: CallbackQuery, state: FSMContext):
    video_id = int(callback.data.split(":")[1])
    await state.set_state(CommentState.waiting_text)
    await state.update_data(video_id=video_id)
    await callback.message.answer("■ ■■■■■■■■ ■■■■■■■■■■:")
    await callback.answer()

@router.message(CommentState.waiting_text)
async def process_comment(message: Message, state: FSMContext):
    data = await state.get_data()
    video_id = data.get("video_id")
    if not video_id:
        await state.clear()
        return

    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            await state.clear()
            return

        # ■■■■■■■■
        ten_min_ago = datetime.utcnow() - timedelta(minutes=10)
        recent = (await session.execute(
            select(func.count(Comment.id)).where(
                Comment.user_id == user.id,
                Comment.created_at >= ten_min_ago
            )
        )).scalar_one()
        if recent >= COMMENTS_PER_10_MIN:
            await message.answer(
                f"■■ ■■ ■■■■■ {COMMENTS_PER_10_MIN} ■■■■■■■■■■ ■■ 10 ■■■■■."
            )
            await state.clear()
            return

        from app.models import Comment as CommentModel
        session.add(CommentModel(
            user_id=user.id,
            video_id=video_id,
            text=message.text
        ))
        user.xp += XP_PER_COMMENT
        await _level_up_check(session, user, message)
        await _update_quest_progress(session, user.id, "comment", 1)

    await message.answer("■ ■■■■■■■■■■ ■■■■■■■■■■!")
    await state.clear()

# =====
# REACTIONS
# =====
@router.callback_query(F.data.startswith("reactions:"))
async def cb_reactions_menu(callback: CallbackQuery):
    video_id = int(callback.data.split(":")[1])
    await callback.message.answer(
        "■■■■■■■■ ■■■■■■■:",
        reply_markup=reaction_menu_keyboard(video_id)
    )

```

[illegible]

```

        user.xp += XP_PER_UPLOAD
        await _level_up_check(session, user, message)
        await _update_quest_progress(session, user.id, "upload", 1)
        await message.answer(
            f"■ ■■■■■ ■{saved.id} ■■■■■■■■■■ ■■ ■■■■■■■■■■!"
        )

@router.message(F.photo)
async def handle_photo_upload(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user or user.status == "banned":
            return
        if not user.agreed_to_rules:
            await message.answer("■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ /start")
            return
        if not user.nickname_set:
            await require_nickname(message, user)
            return

        p = message.photo[-1]
        saved, is_duplicate = await save_photo(
            session, user.id,
            p.file_id, p.file_unique_id,
            p.file_size
        )

        if is_duplicate:
            await message.answer(
                "■■■ ■■■■ ■■■■■ ■■■■ ■■■■■ ■ ■■■■■ ■ ■■ ■■■■■ ■■■■■ ■■■■■■■■■■ ■■■■■■■■■■."
            )
            return

        user.xp += XP_PER_UPLOAD
        await _level_up_check(session, user, message)
        await _update_quest_progress(session, user.id, "upload", 1)
        await message.answer(
            f"■ ■■■■■ ■{saved.id} ■■■■■■■■■■ ■■ ■■■■■■■■■■!"
        )

# =====
# BONUS
# =====
@router.message(F.text == BTN_BONUS)
async def btn_bonus(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return
        ok, result = await claim_daily_bonus(session, user.id)
    if ok:
        await message.answer(
            f"■ ■■■■■ ■■■■■■■■■■: <b>+{result} ■■■■■</b>!",
            parse_mode="HTML"
        )
    else:
        await message.answer(str(result))

# =====
# REFERRALS
# =====
@router.message(F.text == BTN_REFERRALS)
async def btn_referrals(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return
        refs = await count_referrals(session, user.id)

    bot_info = await message.bot.get_me()
    ref_link = f"https://t.me/{bot_info.username}?start={user.referral_code}"
    await message.answer(
        f"■ <b>■■■■■■■■■■</b>\n\n"
        f"■■■■■ ■■■■■■■■■■:\n<code>{ref_link}</code>\n\n"
        f"■■■■■■■■■■: <b>{refs}</b>\n\n"
        f"■■■■■■■■■■: <b>{user.referral_earnings}</b> ■■■■■\n\n"
        f"■■■ ■■■■■■■■■■ ■■■■■■■■■■:\n\n"
        f"• ■■ ■■■■■■■■■■: +{REFERRAL_REWARD_INVITER} ■■■■■\n\n"
        f"• ■■■■■ ■■■■■■■■■■: +{REFERRAL_REWARD_NEW_USER} ■■■■■",
        parse_mode="HTML"
    )

```

```

# =====
# BUY COINS
# =====
@router.message(F.text == BTN_BUY)
async def btn_buy(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return

    # 🎁🎁🎁🎁🎁🎁
    bonus_text = ""
    if DYNAMIC_STAR_DISCOUNT_ENABLED:
        try:
            start_h, end_h = map(int, DYNAMIC_STAR_DISCOUNT_HOURS.split("-"))
            now_h = datetime.utcnow().hour
            if start_h <= now_h < end_h:
                bonus_text = f"\n 🎁🎁🎁🎁🎁🎁 🎁🎁🎁🎁🎁🎁 +{int((DYNAMIC_STAR_DISCOUNT_MULTIPLIER - 1) *
100)}%" 🎁🎁🎁🎁🎁🎁!</b>"
            else:
                bonus_text = f"\n 🎁🎁🎁🎁🎁🎁: {start_h}:00–{end_h}:00 UTC (+{int((DYNAMIC_STAR_DISCOUNT_MULT
IPLIER - 1) * 100)}%)"
        except Exception:
            pass
    bonus_text += f"\n 🎁🎁🎁🎁🎁🎁 🎁🎁🎁🎁🎁🎁: +{FIRST_PURCHASE_DAILY_BONUS} 🎁🎁🎁🎁🎁🎁."

    await message.answer(
        f"🎁 🎁🎁🎁🎁🎁🎁🎁🎁🎁🎁🎁🎁</b>{bonus_text}\n\n🎁🎁🎁🎁🎁🎁🎁🎁🎁🎁:",
        parse_mode="HTML",
        reply_markup=buy_coins_keyboard()
    )

@router.callback_query(F.data.startswith("buy:"))
async def cb_buy_pack(callback: CallbackQuery):
    pack_key = callback.data.split(":")[1]
    pack = STARS_PACKAGES.get(pack_key)
    if not pack:
        await callback.answer("🎁🎁🎁🎁 🎁🎁🎁🎁", show_alert=True)
        return

    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        payment = await create_payment(session, user.id, pack_key)

    await callback.message.answer_invoice(
        title=f"🎁🎁🎁🎁🎁 {pack['title']}",
        description=f"{pack['coins']} 🎁🎁🎁🎁 🎁🎁 {pack['stars']} Stars",
        payload=payment.payload,
        currency="XTR",
        prices=[LabeledPrice(label=pack['title'], amount=pack['stars'])]
    )
    await callback.answer()

@router.callback_query(F.data == "buy_custom")
async def cb_buy_custom(callback: CallbackQuery, state: FSMContext):
    await state.set_state(CustomBuyState.waiting_stars)
    await callback.message.answer("🎁 🎁🎁🎁🎁🎁🎁🎁🎁🎁 Stars (🎁🎁. 1):")
    await callback.answer()

@router.message(CustomBuyState.waiting_stars)
async def process_custom_stars(message: Message, state: FSMContext):
    if not message.text or not message.text.isdigit():
        await message.answer("🎁 🎁🎁🎁🎁🎁🎁🎁🎁🎁.")
        return
    stars = int(message.text)
    if stars < 1:
        await message.answer("🎁 🎁🎁🎁🎁🎁 1 Star.")
        return

    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            await state.clear()
            return
        payment = await create_custom_payment(session, user.id, stars)
        coins = int(stars * STARS_TO_COINS_RATE)

    await message.answer_invoice(

```

```

        title=f"████████ {coins} ██████",
        description=f"{coins} ██████ {stars} Stars",
        payload=payment.payload,
        currency="XTR",
        prices=[LabeledPrice(label=f"{coins} ██████", amount=stars)]
    )
    await state.clear()

@router.pre_checkout_query()
async def pre_checkout(query: PreCheckoutQuery):
    await query.answer(ok=True)

@router.message(F.successful_payment)
async def successful_payment(message: Message):
    payload = message.successful_payment.invoice_payload

    if payload.startswith("vip_"):
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if user:
                now = datetime.utcnow()
                user.vip_until = (
                    user.vip_until + timedelta(days=VIP_DURATION_DAYS)
                    if user.vip_until and user.vip_until > now
                    else now + timedelta(days=VIP_DURATION_DAYS)
                )
                await log_user_action(
                    session, user.id,
                    "buy_vip",
                    f"until={user.vip_until}"
                )
                await session.commit()
            await message.answer(
                f"■ VIP ████████████████████ {VIP_DURATION_DAYS} ██████!"
            )
    elif payload.startswith("promo_"):
        # ████████ {amount} ████████████████████ (██████████)
        parts = payload.split("_")
        if len(parts) >= 5:
            creator_tg_id = int(parts[1])
            amount = int(parts[2])
            uses = int(parts[3])
            hours = int(parts[4])
            async with async_session() as session:
                user = await get_user(session, creator_tg_id)
                if user:
                    promo, cost, error = await create_promocode(
                        session, creator_tg_id,
                        to_decimal(amount), uses, hours
                    )
                    if error:
                        await message.answer(f"■ ████████████████████: {error}")
                    else:
                        bot = await message.bot.get_me()
                        await message.answer(
                            f"■ ████████████████████: \n"
                            f"<code>{promo.code}</code> \n"
                            f"████████: {amount} ████████, ████████████████████: {uses}/{promo.max_uses} \n"
                            f"████████: t.me/{bot.username}?start=promo_{promo.code}",
                            parse_mode="HTML"
                        )
                else:
                    await message.answer("██████████████████.")
            else:
                async with async_session() as session:
                    payment = await apply_successful_payment(session, payload)
                if payment:
                    await message.answer(
                        f"■ ████████████████████! \n"
                        f"██████████████████: <b>{payment.coins_amount}</b> ████████",
                        parse_mode="HTML"
                    )
                else:
                    await message.answer("■ ████████████████████!")

# =====
# OFFERS
# =====
@router.message(F.text == BTN_OFFERS)
async def btn_offers(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):

```

```

        return
    offers = await get_active_offers(session)

if not offers:
    await message.answer("■ ■■■■■■■■ ■■■■■■■■ ■■■■ ■■■■. ■■■■■■■■■■ ■■■■■■!")
    return

kb = InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(
        text="■ ■■■■■■■■ (■■■■■■■■)",
        callback_data="offers_participation"
    )],
    [InlineKeyboardButton(
        text="■ ■■■■■■■■■■■■ ■■■■■■■■■■ ■■■■",
        callback_data="offers_rent_list"
    )],
    [InlineKeyboardButton(
        text="■ ■■■■ ■■■■■■■■",
        callback_data="my_rentals"
    )],
])
await message.answer(
    "■ <b>■■■■■■■■</b></b>\n\n■■■■■■■■■■ ■■■■■■:",
    parse_mode="HTML",
    reply_markup=kb
)

@router.callback_query(F.data == "offers_participation")
async def offers_participation(callback: CallbackQuery):
    async with async_session() as session:
        offers = await get_active_offers(session)

    if not offers:
        await callback.message.answer("■ ■■■■■■■■ ■■■■■■■■ ■■■■.")
        await callback.answer()
        return

    await callback.message.answer(
        "■ <b>■■■■■■■■ ■■■■ ■■■■■■■■■■</b></b>\n\n■■■■■■■■■■ ■■■■■■:",
        parse_mode="HTML",
        reply_markup=offers_list_keyboard(offers)
    )
    await callback.answer()

@router.callback_query(F.data == "back_to_offers")
async def back_to_offers(callback: CallbackQuery):
    await offers_participation(callback)

@router.callback_query(F.data.startswith("offer_open:"))
async def cb_offer_open(callback: CallbackQuery):
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = await get_offer_by_id(session, offer_id)
        if not offer:
            await callback.answer("■■■■■■ ■■ ■■■■■■■■.", show_alert=True)
            return

        from sqlalchemy import select as sa_select
        from app.models import OfferParticipation
        participants = (await session.execute(
            sa_select(func.count(OfferParticipation.id)).where(
                OfferParticipation.offer_id == offer_id
            )
        )).scalar_one()

    text = (
        f"■ <b>{offer.title}</b></b>\n\n"
        f"■ {offer.description}\n\n"
        f"■ ■■■■■■■■■■■■■■■■■■■■: <b>{offer.reward_preview}</b> ■■■■■■\n\n"
        f"■ ■■■■■■ ■■■■■■■■■■■■■■■■■■■■: <b>{offer.reward_final}</b> ■■■■■■\n\n"
        f"■ ■■■■■■■■■■■■■■■■■■■■: {participants}"
    )

    kb_rows = [
        [InlineKeyboardButton(
            text="■ ■■■■■■■■ ■ ■■■■■■",
            url=offer.channel_url
        )],
        [InlineKeyboardButton(
            text="■ ■■■■■■■■■■■■■■■■■■■",
            callback_data=f"offer_start:{offer_id}"
        )],
        [InlineKeyboardButton(
            text="■ ■■■■■■■■■■■■■■■■■■■",
            callback_data=f"offer_check:{offer_id}"
        )],
    ]

```

[illegible]

```

active_count = 0
async with async_session() as session:
    try:
        active_count = (await session.execute(
            select(func.count(OfferRental.id)).where(
                OfferRental.offer_id == o.id,
                OfferRental.status.in_(["active", "pending"])
            )
        )).scalar_one()
    except Exception:
        pass
slots_left = o.max_simultaneous_rentals - active_count
kb_buttons.append([InlineKeyboardButton(
    text=(
        f"■ {o.title[:30]} | "
        f"{o.rent_cost_per_day} ■■■■■/■■■■■ | "
        f"■■■■■■: {slots_left}/{o.max_simultaneous_rentals}"
    ),
    callback_data=f"rent_offer:{o.id}"
)])

kb_buttons.append([InlineKeyboardButton(
    text="■ ■■■■■",
    callback_data="btn_offers_back"
)])

await callback.message.answer(
    text, parse_mode="HTML",
    reply_markup=InlineKeyboardMarkup(inline_keyboard=kb_buttons)
)
await callback.answer()

@router.callback_query(F.data == "btn_offers_back")
async def btn_offers_back(callback: CallbackQuery):
    async with async_session() as session:
        offers = await get_active_offers(session)
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text="■ ■■■■■ (■■■■■■)",
            callback_data="offers_participation"
        )],
        [InlineKeyboardButton(
            text="■ ■■■■■■■■■ ■■■■■■■■■ ■■■■",
            callback_data="offers_rent_list"
        )],
        [InlineKeyboardButton(
            text="■ ■■■ ■■■■■",
            callback_data="my_rentals"
        )],
    ])
    await callback.message.answer(
        "■ <b>■■■■■■</b>\n\n■■■■■■■■ ■■■■■:",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()

@router.callback_query(F.data.startswith("rent_offer:"))
async def rent_offer_start(callback: CallbackQuery, state: FSMContext):
    offer_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        offer = await get_offer_by_id(session, offer_id)
        if not offer or not offer.is_rentable:
            await callback.answer("■■■■■■ ■■■■■■■■■.", show_alert=True)
            return

        active_count = (await session.execute(
            select(func.count(OfferRental.id)).where(
                OfferRental.offer_id == offer_id,
                OfferRental.status.in_(["active", "pending"])
            )
        )).scalar_one()
        slots_left = offer.max_simultaneous_rentals - active_count

    if slots_left <= 0:
        await callback.answer(
            "■ ■■■ ■■■■■ ■■■■■. ■■■■■■■■■ ■■■■■.",
            show_alert=True
        )
        return

    await state.set_state(RentOfferState.waiting_channel_title)
    await state.update_data(offer_id=offer_id)
    await callback.message.answer(
        f"■ <b>■■■■■■ ■■■■ ■: {offer.title}</b>\n\n"
        f"■ ■■■■■■■■■: {offer.rent_cost_per_day} ■■■■■/■■■■■\n"

```



```
f"{offer.title}: {slots_left}/{offer.max_simultaneous_rentals}\n\n"  
f"1/3: {offer.min_rent_days} - {offer.max_rent_days}:",  
parse_mode="HTML"  
)  
await callback.answer()  
  
@router.message(RentOfferState.waiting_channel_title)  
async def rent_channel_title(message: Message, state: FSMContext):  
    title = (message.text or "").strip()  
    if len(title) < 2 or len(title) > 100:  
        await message.answer("❗ Канал должен содержать от 2 до 100 символов.")  
        return  
    await state.update_data(channel_title=title)  
    await state.set_state(RentOfferState.waiting_channel_url)  
    await message.answer(  
        "2/3: Введите ссылку на канал в Telegram (https://t.me/...):"  
    )  
  
@router.message(RentOfferState.waiting_channel_url)  
async def rent_channel_url(message: Message, state: FSMContext):  
    url = (message.text or "").strip()  
    if not (url.startswith("https://t.me/") or url.startswith("t.me/")):  
        await message.answer(  
            "❗ Ссылка должна начинаться с https://t.me/ или t.me/"  
        )  
        return  
    await state.update_data(channel_url=url)  
    await state.set_state(RentOfferState.waiting_days)  
  
    data = await state.get_data()  
    offer_id = data.get("offer_id")  
  
    await message.answer(  
        f"3/3: Вы выбрали предложение № {offer_id}.\n"  
        f"({OFFER_MIN_RENT_DAYS} - {OFFER_MAX_RENT_DAYS}):",  
        reply_markup=rent_days_keyboard(offer_id)  
    )  
  
@router.callback_query(RentOfferState.waiting_days, F.data.startswith("rent_days:"))  
async def rent_days_selected(callback: CallbackQuery, state: FSMContext):  
    parts = callback.data.split(":")  
    offer_id = int(parts[1])  
    days = int(parts[2])  
  
    data = await state.get_data()  
    channel_title = data.get("channel_title", "")  
    channel_url = data.get("channel_url", "")  
  
    async with async_session() as session:  
        offer = await get_offer_by_id(session, offer_id)  
        if not offer:  
            await callback.answer("❌ Предложение не найдено.", show_alert=True)  
            await state.clear()  
            return  
  
        cost = to_decimal(offer.rent_cost_per_day) * days  
        user = await get_user(session, callback.from_user.id)  
        if not user:  
            await callback.answer()  
            await state.clear()  
            return  
  
    kb = InlineKeyboardMarkup(inline_keyboard=[  
        [  
            InlineKeyboardButton(  
                text=f"💰 Общая стоимость: {cost} руб.",  
                callback_data=f"confirm_rent:{offer_id}:{days}"  
            ),  
            InlineKeyboardButton(  
                text="👉",  
                callback_data=f"rent_offer:{offer_id}"  
            ),  
        ],  
    ])  
    await callback.message.answer(  
        f"✅ Подтвердите аренду!  
        f"Предложение: {offer.title}\n"  
        f"Канал: {channel_title}\n"  
        f"Ссылка: {channel_url}\n"  
        f"Длины аренды: {days}\n"  
        f"Стоимость аренды: {cost} руб./сутки.  
        f"Ваш баланс: {user.balance} руб."  
        f"Вы можете отказаться от аренды в любой момент.",  
        parse_mode="HTML",  
        reply_markup=kb
```

```

    )
    await state.update_data(days=days)
    await callback.answer()

@router.callback_query(F.data.startswith("confirm_rent:"))
async def confirm_rent(callback: CallbackQuery, state: FSMContext):
    parts = callback.data.split(":")
    offer_id, days = int(parts[1]), int(parts[2])

    data = await state.get_data()
    channel_title = data.get("channel_title", "")
    channel_url = data.get("channel_url", "")

    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            await state.clear()
            return

        rental, error = await create_offer_rental(
            session,
            offer_id=offer_id,
            user_id=user.id,
            channel_title=channel_title,
            channel_url=channel_url,
            rent_days=days,
        )

    if error:
        await callback.message.answer(error)
        await state.clear()
        await callback.answer()
        return

    await callback.message.answer(
        f"■ <b>■■■■■■ ■■ ■■■■■■ ■■■■■■■■■■!</b>\n\n"
        f"■■■■■: {channel_title}\n"
        f"■■■■■: {days}\n"
        f"■■■■■■■■■: {rental.cost_paid} ■■■■■\n\n"
        f"■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■ ■■■■■ ■■■■■ ■■■■■■■■■ ■■■■■■■■■.\n"
        f"■■■ ■■■■■■■■■ ■■■■■■■■■■■.",
        parse_mode="HTML"
    )
    await state.clear()
    await callback.answer()

@router.callback_query(F.data == "my_rentals")
async def my_rentals(callback: CallbackQuery):
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        rentals = await get_user_rentals(session, user.id)

    if not rentals:
        await callback.message.answer(
            "■ ■■■ ■■■ ■■■■■.\n"
            "■■■■■■■■■■ ■■■■■■■■■ ■■■■ ■ ■■■■■■■ ■ ■■■■■■■!"
        )
        await callback.answer()
        return

    text = "■ <b>■■■ ■■■■■■</b>\n\n"
    for r in rentals:
        offer = await get_offer_by_id(session, r.offer_id)
        offer_name = offer.title if offer else f"##{r.offer_id}"
        status_icon = {
            "pending": "■",
            "active": "■",
            "expired": "■",
            "rejected": "■",
        }.get(r.status, "■")
        expires = r.expires_at.strftime('%d.%m.%Y') if r.expires_at else "-"
        text += (
            f"{status_icon} {offer_name}\n"
            f"■■■■■: {r.renter_channel_title}\n"
            f"■■■■■: {r.rent_days} | ■■■■■■■■■: {r.cost_paid}\n"
            f"■■■■■■: {r.status} | ■■: {expires}\n\n"
        )

    if len(text) > 4000:
        text = text[:4000] + "\n..."
    await callback.message.answer(text, parse_mode="HTML")
    await callback.answer()
```

```
# =====
# GAMES (■ ■■■■■■■■ ■■■■■■■■)
# =====
@router.message(F.text == BTN_GAMES)
async def btn_games(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return

        can_play = await can_play_free_game(session, user.id)
        if can_play:
            await message.answer(
                "■ <b>■■■■■■■■ ■■■■■■■■</b>",
                parse_mode="HTML",
                reply_markup=games_menu_keyboard()
            )
        else:
            kb = InlineKeyboardMarkup(inline_keyboard=[
                [InlineKeyboardButton(
                    text=f"■ ■■■■■■■■ ■■ {GAME_SESSION_COST} ■■■■■",
                    callback_data="game_pay_session"
                )]
            ])
            await message.answer(
                "■■■■■■■■ ■■■■ ■■ ■■■■■■ ■■■■■■■■■■. ■■■■■■■■ ■■■■■■■■.",
                reply_markup=kb
            )

@router.callback_query(F.data == "game_pay_session")
async def game_pay_session(callback: CallbackQuery):
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        # ■■■■■■■■ ■■■■■■■■■■
        if is_admin_or_super(callback.from_user.id, user):
            gs = await increment_game_played(session, user.id) # ■■■■■■■■ ■■■■■■■■
            await callback.answer("■ ■■■■■■■■ ■■■■■■■■■■ (■■■■■■).", show_alert=True)
            return
        ok = await pay_for_game_session(session, callback.from_user.id)
        if ok:
            await callback.answer("■ ■■■■■■■■ ■■■■■■■■■■! ■■■■ 5 ■■■■.", show_alert=True)
        else:
            await callback.answer("■ ■■■■■■■■■■■■■■■■ ■■■■■.", show_alert=True)
    await callback.message.edit_reply_markup(reply_markup=None)

@router.callback_query(F.data == "game_dice")
async def game_dice(callback: CallbackQuery):
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="1 ■■■■■■■■", callback_data="dice_bet:1"),
            InlineKeyboardButton(text="5 ■■■■■■■■", callback_data="dice_bet:5"),
        ],
        [
            InlineKeyboardButton(text="10 ■■■■■■■■", callback_data="dice_bet:10"),
            InlineKeyboardButton(text="25 ■■■■■■■■", callback_data="dice_bet:25"),
        ],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="games_back")],
    ])
    await callback.message.answer(
        "■ <b>■■■■■■■■</b>\n\n4, 5, 6 - ■■■■■■■■ x2!\n\n■■■■■■■■:",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()

@router.callback_query(F.data.startswith("dice_bet:"))
async def dice_bet(callback: CallbackQuery):
    bet = int(callback.data.split(":")[1])
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        if user.balance < to_decimal(bet):
            await callback.answer("■ ■■■■■■■■■■■■■■■■ ■■■■■!", show_alert=True)
            return
        if not is_admin_or_super(callback.from_user.id, user):
            can_play = await can_play_free_game(session, user.id)
```

```

        if not can_play:
            await callback.answer("██████████ ████ ██████████.", show_alert=True)
            return

dice_msg = await callback.message.answer_dice(emoji="🎲")
dice_value = dice_msg.dice.value

user.balance -= to_decimal(bet)
if not is_admin_or_super(callback.from_user.id, user):
    await increment_game_played(session, user.id)

if dice_value >= 4:
    win = to_decimal(bet) * 2
    user.balance += win
    net = win - to_decimal(bet)
    result_text = f"🎲 ████████: {dice_value}\n🎲 ██████████! +{win} ██████"
else:
    net = -to_decimal(bet)
    result_text = f"🎲 ████████: {dice_value}\n🎲 ██████████ -{bet} ██████"

user.xp += XP_PER_GAME
await _level_up_check(session, user, callback)

session.add(GameHistory(
    user_id=user.id,
    game_type="dice",
    bet=to_decimal(bet),
    result=net,
    details=f"dice={dice_value}"
))
await log_balance_change(
    session, user, net, "game_dice",
    details=f"bet={bet}, dice={dice_value}"
)
await session.commit()

await callback.message.answer(
    f"{result_text}\n🎲 ████████: {user.balance}"
)
await callback.answer()

@router.callback_query(F.data == "game_coinflip")
async def game_coinflip(callback: CallbackQuery):
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="1 ██████", callback_data="coinflip_bet:1"),
            InlineKeyboardButton(text="5 ██████", callback_data="coinflip_bet:5"),
        ],
        [
            InlineKeyboardButton(text="10 ██████", callback_data="coinflip_bet:10"),
            InlineKeyboardButton(text="25 ██████", callback_data="coinflip_bet:25"),
        ],
        [InlineKeyboardButton(text="🎲 ██████", callback_data="games_back")],
    ])
    await callback.message.answer(
        "🎲 <b>██████/██████</b>\n\n50/50 █████ x2!\n\n██████:",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()

@router.callback_query(F.data.startswith("coinflip_bet:"))
async def coinflip_bet(callback: CallbackQuery):
    bet = int(callback.data.split(":")[1])
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        if user.balance < to_decimal(bet):
            await callback.answer("🎲 ██████████████████████████!", show_alert=True)
            return
        if not is_admin_or_super(callback.from_user.id, user):
            can_play = await can_play_free_game(session, user.id)
            if not can_play:
                await callback.answer("██████████ ████ ██████████.", show_alert=True)
                return

    coin_msg = await callback.message.answer_dice(emoji="🎲")
    won = coin_msg.dice.value >= 4

    user.balance -= to_decimal(bet)
    if not is_admin_or_super(callback.from_user.id, user):
        await increment_game_played(session, user.id)
    user.xp += XP_PER_GAME

```

```

    if won:
        win = to_decimal(bet) * 2
        user.balance += win
        net = win - to_decimal(bet)
        result_text = f"🎰 🎰🎰🎰! 🎰 +{win} 🎰🎰🎰"
    else:
        net = -to_decimal(bet)
        result_text = f"🎰 🎰🎰🎰! 🎰 -{bet} 🎰🎰🎰"

    await _level_up_check(session, user, callback)
    session.add(GameHistory(
        user_id=user.id,
        game_type="coinflip",
        bet=to_decimal(bet),
        result=net,
        details=f"won={won}"
    ))
    await log_balance_change(
        session, user, net, "game_coinflip",
        details=f"bet={bet}, won={won}"
    )
    await session.commit()

    await callback.message.answer(
        f"{result_text}\n🎰 🎰🎰🎰: {user.balance}"
    )
    await callback.answer()

@router.callback_query(F.data == "game_guess")
async def game_guess(callback: CallbackQuery):
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="1 🎰🎰🎰", callback_data="guess_bet:1"),
            InlineKeyboardButton(text="5 🎰🎰🎰", callback_data="guess_bet:5"),
            InlineKeyboardButton(text="10 🎰🎰🎰", callback_data="guess_bet:10"),
        ],
        [InlineKeyboardButton(text="🎰 🎰🎰🎰", callback_data="games_back")],
    ])
    await callback.message.answer(
        "🎰 <b>🎰🎰🎰 🎰🎰🎰</b>\n\n🎰🎰🎰 1-6 - x5 🎰🎰🎰!\n\n🎰🎰🎰:",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()

@router.callback_query(F.data.startswith("guess_bet:"))
async def guess_bet_start(callback: CallbackQuery, state: FSMContext):
    bet = int(callback.data.split(":")[1])
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        if user.balance < to_decimal(bet):
            await callback.answer("🎰 🎰🎰🎰🎰🎰 🎰🎰!", show_alert=True)
            return
        if not is_admin_or_super(callback.from_user.id, user):
            can_play = await can_play_free_game(session, user.id)
            if not can_play:
                await callback.answer("🎰🎰🎰🎰 🎰🎰 🎰🎰🎰🎰.", show_alert=True)
                return

    await state.update_data(guess_bet=bet)
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="1", callback_data="guess_num:1"),
            InlineKeyboardButton(text="2", callback_data="guess_num:2"),
            InlineKeyboardButton(text="3", callback_data="guess_num:3"),
        ],
        [
            InlineKeyboardButton(text="4", callback_data="guess_num:4"),
            InlineKeyboardButton(text="5", callback_data="guess_num:5"),
            InlineKeyboardButton(text="6", callback_data="guess_num:6"),
        ],
    ])
    await callback.message.answer("🎰🎰🎰 🎰🎰:", reply_markup=kb)
    await callback.answer()

@router.callback_query(F.data.startswith("guess_num:"))
async def guess_num(callback: CallbackQuery, state: FSMContext):
    guess = int(callback.data.split(":")[1])
    data = await state.get_data()
    bet = data.get("guess_bet", 1)

    async with async_session() as session:

```

```

    user = await get_user(session, callback.from_user.id)
    if not user:
        await callback.answer()
        return
    if user.balance < to_decimal(bet):
        await callback.answer("❌ 🎰🎰🎰🎰🎰🎰! ", show_alert=True)
        return
    if not is_admin_or_super(callback.from_user.id, user):
        can_play = await can_play_free_game(session, user.id)
        if not can_play:
            await callback.answer("❌ 🎰🎰🎰🎰🎰🎰.", show_alert=True)
            return

    dice_msg = await callback.message.answer_dice(emoji="🎲")
    actual = dice_msg.dice.value

    user.balance -= to_decimal(bet)
    if not is_admin_or_super(callback.from_user.id, user):
        await increment_game_played(session, user.id)
    user.xp += XP_PER_GAME

    if guess == actual:
        win = to_decimal(bet) * 5
        user.balance += win
        net = win - to_decimal(bet)
        result_text = f"🎲 🎰🎰🎰🎰 {actual}! 🎲🎰🎰🎰🎰! 🎲 +{win} 🎰🎰🎰🎰"
    else:
        net = -to_decimal(bet)
        result_text = f"🎲 🎰🎰🎰🎰 {actual}, 🎲🎰🎰🎰🎰 {guess}. 🎲 -{bet} 🎰🎰🎰🎰"

    await _level_up_check(session, user, callback)
    session.add(GameHistory(
        user_id=user.id,
        game_type="guess",
        bet=to_decimal(bet),
        result=net,
        details=f"guess={guess}, actual={actual}"
    ))
    await log_balance_change(
        session, user, net, "game_guess",
        details=f"bet={bet}, guess={guess}, actual={actual}"
    )
    await session.commit()

    await callback.message.answer(
        f"{result_text}\n🎲 🎰🎰🎰🎰: {user.balance}"
    )
    await state.clear()
    await callback.answer()

@router.callback_query(F.data == "games_back")
async def games_back(callback: CallbackQuery):
    await callback.message.answer(
        "🎲 🎰🎰🎰:",
        reply_markup=games_menu_keyboard()
    )
    await callback.answer()

# =====
# TOPS
# =====
@router.message(F.text == BTN_TOPS)
async def btn_tops(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return
    await message.answer(
        "🎲 <b>🎰🎰🎰</b>",
        parse_mode="HTML",
        reply_markup=tops_menu_keyboard()
    )

@router.callback_query(F.data == "top_uploaders")
async def top_uploaders(callback: CallbackQuery):
    async with async_session() as session:
        rows = (await session.execute(
            select(User, func.count(Video.id).label("cnt"))
                .join(Video, Video.uploader_user_id == User.id)
                .where(Video.status == "approved")
                .group_by(User.id)
                .order_by(desc("cnt"))
                .limit(10)
        ))

```

```

   )).all()

text = "■ <b>■■■ ■■■■■■■■■■</b>\n\n"
medals = ["■", "■", "■"]
seen, rank = set(), 0
for u, cnt in rows:
    if u.id in seen:
        continue
    seen.add(u.id)
    rank += 1
    icon = medals[rank - 1] if rank <= 3 else f"{rank}."
    text += f"{icon} {get_display_name(u)} – {cnt} ■■■■■\n"
if not rows:
    text += "■■■■■"
await callback.message.answer(text, parse_mode="HTML")
await callback.answer()

@router.callback_query(F.data == "top_viewers")
async def top_viewers(callback: CallbackQuery):
    async with async_session() as session:
        rows = (await session.execute(
            select(User, func.count(VideoView.id).label("cnt"))
            .join(VideoView, VideoView.user_id == User.id)
            .group_by(User.id)
            .order_by(desc("cnt"))
            .limit(10)
        )).all()

    text = "■ <b>■■■ ■■■■■■■■■■</b>\n\n"
    medals = ["■", "■", "■"]
    seen, rank = set(), 0
    for u, cnt in rows:
        if u.id in seen:
            continue
        seen.add(u.id)
        rank += 1
        icon = medals[rank - 1] if rank <= 3 else f"{rank}."
        text += f"{icon} {get_display_name(u)} – {cnt} ■■■■■■■■■■\n"
    if not rows:
        text += "■■■■■"
    await callback.message.answer(text, parse_mode="HTML")
    await callback.answer()

@router.callback_query(F.data == "top_levels")
async def top_levels(callback: CallbackQuery):
    async with async_session() as session:
        users = (await session.execute(
            select(User).order_by(desc(User.xp)).limit(10)
        )).scalars().all()

    text = "■ <b>■■■ ■■ XP</b>\n\n"
    medals = ["■", "■", "■"]
    seen, rank = set(), 0
    for u in users:
        if u.id in seen:
            continue
        seen.add(u.id)
        rank += 1
        icon = medals[rank - 1] if rank <= 3 else f"{rank}."
        text += f"{icon} {get_display_name(u)} – ■■.{u.level} ({u.xp} XP)\n"
    if not users:
        text += "■■■■■"
    await callback.message.answer(text, parse_mode="HTML")
    await callback.answer()

@router.callback_query(F.data == "top_richest")
async def top_richest(callback: CallbackQuery):
    async with async_session() as session:
        users = (await session.execute(
            select(User).order_by(desc(User.balance)).limit(10)
        )).scalars().all()

    text = "■ <b>■■■ ■■■■■■■■■■</b>\n\n"
    medals = ["■", "■", "■"]
    seen, rank = set(), 0
    for u in users:
        if u.id in seen:
            continue
        seen.add(u.id)
        rank += 1
        icon = medals[rank - 1] if rank <= 3 else f"{rank}."
        text += f"{icon} {get_display_name(u)} – {u.balance:.2f} ■■■■■\n"
    if not users:
        text += "■■■■■"
    await callback.message.answer(text, parse_mode="HTML")

```

```

await callback.answer()

# =====
# QUESTS
# =====
@router.message(F.text == BTN_QUESTS)
async def btn_quests(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return
        if not await require_nickname(message, user):
            return

        today = datetime.utcnow().date()
        quests = (await session.execute(
            select(DailyQuestProgress).where(
                DailyQuestProgress.user_id == user.id,
                DailyQuestProgress.quest_date == today
            )
        )).scalars().all()

        if not quests:
            quest_list = DAILY_QUESTS.copy()
            if is_vip(user):
                quest_list += PREMIUM_DAILY_QUESTS
            for q in quest_list:
                session.add(DailyQuestProgress(
                    user_id=user.id,
                    quest_type=q["type"],
                    quest_date=today,
                    progress=0,
                    target=q["target"],
                    reward=to_decimal(q["reward"]),
                ))
            await session.commit()
            quests = (await session.execute(
                select(DailyQuestProgress).where(
                    DailyQuestProgress.user_id == user.id,
                    DailyQuestProgress.quest_date == today
                )
            )).scalars().all()

        text = "■ <b>■■■■■■■■■■ ■■■■■■</b>\n\n"
        for q in quests:
            status = "■" if q.completed else "□"
            claimed = " ✓ ■■■■■■■■" if q.reward_claimed else ""
            text += (
                f"{status} {q.quest_type}: "
                f"{q.progress}/{q.target} - "
                f"{q.reward} ■■■■■■{claimed}\n"
            )

        await message.answer(
            text,
            parse_mode="HTML",
            reply_markup=quests_keyboard(quests)
        )

@router.callback_query(F.data.startswith("quest_claim:"))
async def quest_claim(callback: CallbackQuery):
    quest_id = int(callback.data.split(":")[1])
    async with async_session() as session:
        quest = (await session.execute(
            select(DailyQuestProgress).where(DailyQuestProgress.id == quest_id)
        )).scalar_one_or_none()
        if not quest:
            await callback.answer("■■■■■ ■■ ■■■■■■.", show_alert=True)
            return
        if not quest.completed:
            await callback.answer("■■■■■ ■■■ ■■ ■■■■■■■■!", show_alert=True)
            return
        if quest.reward_claimed:
            await callback.answer("■■■■■■■ ■■■ ■■■■■■■■!", show_alert=True)
            return

    user = await get_user_by_id(session, quest.user_id)
    if not user:
        await callback.answer()
        return

    quest.reward_claimed = True
    await log_balance_change(
        session, user, quest.reward,
        "quest_reward", source_id=quest.id
    )

```



```

        user.balance += quest.reward
        await session.commit()

    await callback.answer(f"■ ■■■■■■■■ {quest.reward} ■■■■■■!", show_alert=True)

@router.callback_query(F.data.startswith("quest_done:"))
async def quest_done(callback: CallbackQuery):
    await callback.answer("■ ■■■■■■ ■■■■■■■■, ■■■■■■■■ ■■■ ■■■■■■■■.", show_alert=True)

@router.callback_query(F.data.startswith("quest_info:"))
async def quest_info(callback: CallbackQuery):
    await callback.answer("■ ■■■■■■ ■■■ ■■ ■■■■■■■■. ■■■■■■■■■■!", show_alert=True)

# =====
# HELPER: ■■■■■■■■■■ ■■■■■■■■
# =====
async def _update_quest_progress(
    session,
    user_id: int,
    quest_type: str,
    amount: int = 1
):
    today = datetime.utcnow().date()
    quests = (await session.execute(
        select(DailyQuestProgress).where(
            DailyQuestProgress.user_id == user_id,
            DailyQuestProgress.quest_type == quest_type,
            DailyQuestProgress.quest_date == today,
            DailyQuestProgress.completed == False
        )
    )).scalars().all()

    for quest in quests:
        quest.progress = min(quest.progress + amount, quest.target)
        if quest.progress >= quest.target:
            quest.completed = True

    try:
        await session.commit()
    except Exception:
        await session.rollback()

# =====
# ■■■■■■ ■■■■■■■■ – FORCED OFFER
# =====

@router.callback_query(F.data == "forced_offer_wait")
async def forced_offer_wait(callback: CallbackQuery):
    await callback.answer(
        f"■ ■■■■■■■■ ■■■ ■■■■■■■■...",
        show_alert=False
    )

@router.callback_query(F.data.startswith("forced_offer_continue:"))
async def forced_offer_continue(callback: CallbackQuery):
    offer_id = int(callback.data.split(":")[1])

    async with async_session() as session:
        offer = await get_offer_by_id(session, offer_id)
        if offer:
            kb = InlineKeyboardMarkup(inline_keyboard=[
                [InlineKeyboardButton(
                    text="■ ■ ■■■■■■■■■■ – ■■■■■■■■ ■■■■■■■■",
                    callback_data=f"offer_start:{offer_id}"
                )],
                [InlineKeyboardButton(
                    text="■ ■ ■■■■■■■■■■ ■■■■■■■■",
                    callback_data="watch_video_content"
                )],
            ])
            await callback.message.answer(
                f"■ ■■■■■■■■, ■■ ■■■■■■■■■■ ■■ <b>{offer.title}</b> "
                f"■■■■■■■ ■■■■■■■■■■ <b>{offer.reward_preview} ■■■■■■</b>!\n"
                f"■■■■■■■■■ ■■■■■■■■■■?",
                parse_mode="HTML",
                reply_markup=kb
            )
        else:
            await watch_video_content(callback)
            return

    await callback.answer()

```

[illegible]

```

amount = data["promo_amount"]
uses = data["promo_uses"]
star_cost = calculate_promocode_star_cost(to_decimal(amount), uses)

async with async_session() as session:
    user = await get_user(session, message.from_user.id)
    if not user:
        await state.clear()
        return
    admin_free = is_admin_or_super(message.from_user.id, user)
    if admin_free:
        promo, _, error = await create_promocode(session, message.from_user.id,
                                                to_decimal(amount), uses, hours,
                                                admin_free=True)

        if error:
            await message.answer(error)
        else:
            await message.answer(
                f"■ ■■■■■■■■ ■■■■■■ (■■■■■-■■■■■):\n"
                f"<code>{promo.code}</code>\n"
                f"■■■■■: {amount} ■■■■■, ■■■■■■■■■■: {uses}/{promo.max_uses}\n"
                f"■■■■■: t.me/{(await message.bot.get_me()).username}?start=promo_{promo.code}",
                parse_mode="HTML"
            )
            await state.clear()
            return

# VIP ■■■■■■■■■■
if is_vip(user) and user.promo_created_this_month < VIP_FREE_PROMO_PER_MONTH:
    promo, cost, error = await create_promocode(session, message.from_user.id,
                                                to_decimal(amount), uses, hours)

    if error:
        await message.answer(error)
    else:
        await message.answer(
            f"■ ■■■■■■■■ VIP-■■■■■■■■■:\n"
            f"<code>{promo.code}</code>\n"
            f"■■■■■: {amount} ■■■■■, ■■■■■■■■■■: {uses}\n"
            f"■■■■■■■■■ ■■■■■■■■■■ ■ ■■■■ ■■■■■■: {VIP_FREE_PROMO_PER_MONTH - user.promo_created_t
his_month}",
            parse_mode="HTML"
        )
        await state.clear()
        return

# ■■■■■■■■ - ■■■■■■■■■■ ■■■■■■
payload = f"promo_{message.from_user.id}_{amount}_{uses}_{hours}_{uuid.uuid4().hex[:4]}"
await message.answer_invoice(
    title="■■■■■■■■■ ■■■■■■■■■■",
    description=f"{amount} ■■■■■ x {uses} ■■■■. ■■ {hours}■",
    payload=payload,
    currency="XTR",
    prices=[LabeledPrice(label="■■■■■■■■■", amount=star_cost)]
)
await state.clear()

@router.callback_query(F.data == "promo_activate")
async def promo_activate_start(callback: CallbackQuery, state: FSMContext):
    await state.set_state(PromoCreateState.waiting_amount) # ■■■■■■■■■■ ■■■■ ■■■■■■■■■■ ■■■■■■■■■■
    await callback.message.answer("■■■■■■■■■ ■■■■■■■■■■:")
    await callback.answer()

# ■■■■■■■■■■ ■■■■■■ ■■■■ ■■■■■■■■■■ (■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■ waiting_amount)
@router.message(PromoCreateState.waiting_amount)
async def promo_activate_code(message: Message, state: FSMContext):
    # ■■■■■■■■■■, ■■■■ ■■ ■■■■■■ ■ ■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
    code = message.text.strip()
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            await state.clear()
            return
        result = await activate_promocode(session, user.id, code)
        await message.answer(result)
    await state.clear()

@router.callback_query(F.data == "promo_my")
async def promo_my(callback: CallbackQuery):
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        if not user:
            await callback.answer()
            return
        promos = (await session.execute(
            select(Promocode).where(Promocode.creator_user_id == user.id)

```

```

        .order_by(desc(Promocode.created_at)).limit(10)
    )).scalars().all()
    if not promos:
        await callback.message.answer("■ ■■■ ■■■■ ■■■ ■■■■■■■■■■.")
        await callback.answer()
        return
    text = "■ <b>■■■■ ■■■■■■■■■■:</b>\n\n"
    for p in promos:
        status = "■" if p.is_active else "■"
        text += (
            f"{status} <code>{p.code}</code>\n"
            f"■■■■: {p.coin_amount} | ■■■: {p.used_count}/{p.max_uses}\n"
            f"■■: {p.expires_at.strftime('%d.%m %H:%M')} if p.expires_at else '∞'}\n\n"
        )
    await callback.message.answer(text, parse_mode="HTML")
    await callback.answer()

```

```

=====
FILE: requirements.txt
=====

```

```

aiogram==3.15.0
aiohttp>=3.9.0,<3.11
sqlalchemy==2.0.36
aiosqlite==0.20.0
asynccpg==0.29.0
python-dotenv==1.0.1
reportlab==4.2.2

```

```

=====
FILE: TODO.md
=====

```

```

# Bug Fixes - Print to Logger Cleanup

```

```

## Progress

```

- [x] Analyze codebase and identify bugs (prints, no logger)
- [x] Create edit plan and get approval

```

## Tasks

```

- [x] Edit app/make_project_pdf.py: Replace all print() with logger calls, add setup_logging(), wrap main() try/except
- [x] Edit app/user_handlers.py: Replace print("\nSTART COMMAND RECEIVED\n")
- [x] Edit app/migrate.py: Replace print("\nMigrations applied successfully\n")
- [x] Test: run python app/make_project_pdf.py (check PDF + logs no prints) - SUCCESS: PDF generated with JSON logs, no prints
- [x] Test: python app/migrate.py (logs) - SUCCESS: Logger setup, runs (migrate error expected if no DB tables)
- [] Run bot, /start (no print)
- [] attempt_completion

```

**Estimated fixes: 3 files, cleanup for prod-ready**

```